Supplementary Material for "RAI: Pause-Free Committee Changes and Fork Recovery for Consensusless Payments"

Anonymous Author(s)

Regular paper (supplement, not part of the page budget)

This supplement complements the submission. Appendices A to C give the formal statement of the protocol that Sections III to VII of the main text describe in words: the model and the two abstract services with their properties C1–C2 and A1–A4, the account-local rules, and the epoch transition with the report format, eligibility, the construction BuildState, and installation. Appendix D gives the full proofs of the results of Section VIII of the main text and the counterexample behind the early-vote rule. Appendix E describes the TLA$^+$ models, their abstractions, configurations, and results. Appendix F lists the scripted regression cases derived from the models and their status in the prototype. Appendix G records the prototype's departures from the protocol, implementation notes, the additional experiments (committee change, restart and durable signing, handoff), and how to reproduce the runs. Theorem and lemma numbers are those of the main text; Theorem 3 of the main text is Theorems 3 and 4 here. Throughout, $N = 3f + 2p + 1$, $q = N - f - p$, $r = f + p + 1$, and reports are selected in sets of $N - f$.

Appendix A
Formal Model, Assumptions, and Abstract Services

This appendix and the two that follow are the formal counterpart of Sections III to VII of the main text: the same rules, stated with the notation the proofs use.

A. Participants, faults, and retained evidence

Clients own accounts and sign blocks. An authenticated committee contains $N = 3f + 2p + 1$ equal-weight validator identities, $f, p \geq 0$, of which at most f are Byzantine. Corruption is static: adversarial identities are fixed throughout the execution, and a correct departing identity is not later compromised. Membership admission, Sybil resistance, and the per-committee fault bound are deployment assumptions.

Signatures are unforgeable and hashes collision resistant; canonical encodings bind the network session and message domain. An account vote binds its epoch, committee, account, slot, vote type, block hash, and admission-witness digest, where the witness identifies the anchor and evidence under which the vote was admitted, including its base checkpoint (Section B-B); checkpoint votes bind the handoff context and deciding committee. Distinct signatures by one identity never contribute twice to a quorum for the same block.

TABLE I
Account and closure thresholds. The example has $f = p = 1$, $N = 6$.

Evidence	Threshold	Example
Notarization NC / exclusion witness XW	$q = N - f - p$	4
Normal finality FC	q final votes	4
Fast finality FF	$N - p$ settled first votes	5
Recovery support	$r = f + p + 1$	3
Selected reports	$N - f$	5

Correct validators persist their first vote and terminal state before releasing signatures. They retain blocks, signed votes, and finite dependency evidence, and relay them after local voting ends; a certificate is a deterministic local view of sufficient individually verified votes, not a separately gossiped object. A retained item held by a correct validator eventually reaches every correct validator that needs it (the *data service*), and departing validators continue service until correct successors hold durable copies.

B. Quorums, accounts, and canonical state

The account path uses $N = 3f + 2p + 1$, $q = 2f + p + 1$, and $r = f + p + 1$ (Table I). The parameter p is the number of first-vote responses the fast path can omit, not an additional fault allowance. We repeatedly use

$$2q - N = f + 1, \quad q - 2f = p + 1, \quad (N - p) - 2f = r. \tag{1}$$

Each account is a tree of owner-signed blocks rooted at genesis. A block's slot is one greater than its parent's slot; slots are account positions, not times. Blocks of one account conflict when neither is an ancestor of the other. An owner equivocates by signing conflicting blocks; safety covers such owners, liveness does not promise a winner under persistent contention.

A closed checkpoint is $S_e = (L_e, \Sigma_e)$. The ledger L_e identifies finalized prefixes, retained unresolved branches, and lock records. A lock record names its block, its strength (notarization or recovery), and its *origin*: the epoch whose potential finality it protects. A block may carry several recovery records, at most one per origin. Each record is discharged separately, and only by evidence of its own origin epoch; the block stays retained while any of its records survives. The application state

Σ_e contains balances, received-send identifiers, and applied-operation identifiers. A block is final either from an eligible account FC/FF or because a decided checkpoint promoted it under Section C-E; an undecided candidate has no finality effect.

Let F_e be the finalized block set of L_e. A validator also maintains a certified live overlay O_i of locally finalized blocks not yet in F_e; its effective final set is $F_e \cup O_i$, and its live view is the deterministic replay of that set. Section C-F defines installation without rolling back or reissuing effects. A deterministic function derives the committee C_e from Σ_e, never from a private overlay.

C. Latent finality and progress assumptions

For a fixed epoch and admission context, a block is *latent-final* at a cut if released signatures and valid dependency evidence suffice to verify an eligible FC or FF for it or for a descendant selecting it; *late* finality is such a proof assembled or exposed after reports freeze or after a checkpoint is decided. A closure obligation also covers a certificate completed by subsequent old-epoch signatures from identities that have not stopped. Safety has no message-delay bound. Progress requires finite relevant admitted work, fair processing and retry, retained-data availability, and recurring sufficiently long bounded-delay windows in which δ bounds delivery and processing and clock rates lie in $[1-\rho, 1+\rho]$. An uncontested continuation has no conflicting owner-signed alternative at any new position, including signatures created earlier.

D. The two abstract services

RAI commits to sets by their roots and agrees on candidates by their digests. Two services turn these commitments into shared state; RAI states their interfaces and the properties it uses, not their mechanics.

Convergence. A reporter calls $\mathsf{Freeze}_i(\mathsf{ctx}_e, T_i, G_i)$, which returns the signed header R_i of Section C-B and retains the frozen sets. Any validator calls $\mathsf{Reconstruct}_j(R_i)$, which returns (T, G) or is still pending.

C1 (Integrity). If $\mathsf{Reconstruct}_j(R_i)$ returns (T, G) at a correct j, then $\mathsf{Root}(T)$ and $\mathsf{Root}(G)$ are the roots signed in R_i. With collision resistance, $(T, G) = (T_i, G_i)$ when i is correct, and a Byzantine i is bound to one pair per header.

C2 (Termination). If a correct i froze R_i and R_i reaches a correct j, then under the data service and fair retry $\mathsf{Reconstruct}_j(R_i)$ eventually returns.

Full transfer of the committed sets, an authenticated difference against a shared projection, and set reconciliation all satisfy C1 and C2; they differ in cost. A Byzantine reporter can make only its own report unreconstructible.

Consensus. For handoff e, members of the closing committee K_e call $\mathsf{Propose}(\mathsf{ctx}_e, v)$ with candidates $v = (Q, \mu, d)$ of Section C-D, and $\mathsf{Decide}(\mathsf{ctx}_e)$ returns a value v with a proof π_e that $\mathsf{Verify}(\mathsf{ctx}_e, v, \pi_e)$ accepts at any party knowing K_e. RAI supplies the predicate $\mathsf{Eligible}_e(v)$ of Section C-D, which a correct member can evaluate once it has reconstructed the reports v names.

A1 (Agreement). Two accepted proofs for ctx_e carry the same v.

A2 (Eligibility). An accepted proof for ctx_e carries a v with $\mathsf{Eligible}_e(v)$: a correct member supports a value only after checking $\mathsf{Eligible}_e$ on reconstructed inputs, and no decision forms without correct support.

A3 (Integrity). A proof cannot be replayed for another context, and Verify needs only K_e and the retained inputs v names.

A4 (Termination). If at least $N - f$ correct members of K_e eventually hold some eligible candidate for ctx_e, then every correct member eventually decides, given recurring prepared windows long enough for the service's own rounds.

A2 is external validity [1] with a predicate the service treats as opaque. C2 feeds A4: convergence makes correct reports usable, usable reports make an eligible candidate constructible at every correct member, and the consensus service then decides it. Nothing in A1–A4 refers to accounts, blocks, or report contents; any Byzantine agreement protocol with external validity and verifiable decisions, leader-based or not, fits the interface. Safety (Appendix D) uses C1 and A1–A3 together with the static fault bound, authentication, persistent voting, and the voting rules of Section B; termination additionally uses C2, A4, the data service, finite preparation, and recurring prepared windows.

APPENDIX B
FORMAL ACCOUNT-LOCAL RULES

A. Blocks, operations, and validation

A block is $B = (a, h_p, x, \sigma_a)$: account, parent hash, ordered operation batch, and owner signature, with identifier $h(B) = H(\mathsf{Block}, a, h_p, x)$. The epoch is absent from the identifier, so that a nonfinal omitted block can be retried with new-epoch votes; an empty batch is permitted for owner-authorized branch continuation. The operations are $\mathsf{Send}(b, z)$, for destination b and positive amount z, and $\mathsf{Receive}(s)$, where $s = (h(B), j)$ identifies a send at operation index j. A send debits its owner; a receive credits only the named destination and requires an explicit proof that the send is already final.

Validation replays the entire unresolved selected path from its finalized anchor, then the proposed batch, checking signatures, consecutive parent slots, amounts, balances, and duplicate receives; it has no application effect, and every finalization repeats the checks and applies only unapplied operations. A finality proof is an account-certificate proof with the evidence needed to verify eligibility, or a verified decided checkpoint that promoted the block. A proof used for epoch-e voting cannot depend on S_e or a later checkpoint, and a receive's send proof must be available before a correct validator supports the receiving path, which makes replay dependencies well founded (Lemma 7).

B. Single-support voting and durable state

An account instance is $(session, e, a, v)$. A correct validator first-votes at most once in that instance, for the first admissible available block it accepts; there is no second-look support for a rival and no account timeout vote.

Definition 1 (Support, notarization certificate, exclusion witness): A member of K_e *supports* a block B in an instance if it signed an epoch-e first vote for B or an epoch-e late notarization for B; a correct validator supports at most one block per instance. A *notarization certificate* $\mathsf{NC}_e(B)$ is a set of q epoch-e *first votes* for B from distinct members. An *exclusion witness* $\mathsf{XW}_e(B)$ is a set of q epoch-e supports for B from distinct members, first votes or late notarizations in any mix. Every $\mathsf{NC}_e(B)$ is an $\mathsf{XW}_e(B)$; the converse fails.

An NC of first votes is what makes a block complete, what a validator records before final-voting, what an N report entry claims, and what Rules 1 and 3 of Section C-E consume; an XW serves only as the closing-epoch evidence of the overlap exception, as the discharge of a recovery record of origin e on a conflicting block, and inside the overlap certificate behind an F claim. Letting late notarizations count toward an NC makes correct reports unusable (Appendix E).

A block is *complete* in epoch e when its data and required ancestry are available and the validator has assembled its NC_e. A complete eligible block is durably recorded as notarized before the validator final-votes, uses it as a complete parent, or freezes its report. The validator can final-vote only for its own first-voted block, and atomically records a terminal flag with that signature.

Certificates and the early-vote rule. Every first vote names its *base*, the last checkpoint its signer had installed when signing; the admission-witness digest binds it. An epoch-e first vote is *settled* if its base is S_{e-1} and *early* otherwise. An eligible block finalizes on an $\mathsf{FC}_e(B)$ of q final votes or an $\mathsf{FF}_e(B)$ of $N - p$ *settled* first votes; early first votes count toward $\mathsf{NC}_e(B)$ and $\mathsf{XW}_e(B)$ and toward nothing else. A validator that has not installed S_{e-1} releases an epoch-e final vote for B only while it holds, for B and every unresolved block of its selected prefix, the closing-epoch exclusion witness and the current-epoch NC that the overlap exception requires, and it retains them with its notarization record; after installing S_{e-1} it final-votes under ordinary rechecking. A block first-voted during a close therefore finalizes in two rounds rather than one. Without the rule, a fast certificate can be formed from early votes whose eligibility rests on a witness their signers never held, and the next closure cannot see it (Appendix E).

A finalized block's selected unresolved ancestors finalize with it, and incompatible provisional branches leave the effective live view; an NC alone has no balance effect.

Cross-epoch voting record. A correct identity stops epoch-$(e-1)$ first and final voting before opening epoch e. While S_{e-1} is unknown, it does not first-vote for a different block at a position where it supported a block in epoch $e-1$; re-voting the same block is allowed. This follows from the second termination rule below, since such a position is contested for it; we state it separately because the models of Appendix E that predate the termination rules build it in, and they stay safe without it (`Overlap_noCross`, `Handoff_same_noCross`, `Prefix4_shared1_noCross`, and the $N = 6$ simulations of `RaiPrefix`). Once the predecessor is installed, its finalized history, frontier, and carried lock records govern the position.

Termination rules. Two further rules make every forked position final (Theorem 4); both only withhold votes. *Checkpoints decide:* once a validator has installed a checkpoint that holds an unresolved block K at (a, v), it casts no new first, late, or final vote at (a, v), or at a later position of account a, for a block whose branch bypasses a surviving record of that checkpoint, unless the bypassing block at the divergent position is final at the validator. An exclusion witness that discharges the record still makes the rival admissible as an instance, lets the validator accept a rival's overlap certificate (Lemma 8), and discharges the record inside a manifest; it earns the rival no vote. *No early votes at a contested position:* a position is *contested* at a validator once it has seen two conflicting blocks there or its latest installed checkpoint holds an unresolved block there; at a contested position a validator casts no new epoch-e first vote before it has installed S_{e-1}. Re-broadcasting a vote already cast is allowed.

Late notarization. A member of K_{e-1} keeps notarizing for its closing epoch until it installs S_{e-1}. When it first-votes B at (a, v) in epoch e while S_{e-1} is unknown, it also signs a notarization-only epoch-$(e-1)$ vote for B, unless it supported a different block at (a, v) in epoch $e - 1$; if it first-voted B itself there, that vote already supports B. The late vote is durably recorded before release. It counts toward an epoch-$(e - 1)$ exclusion witness and toward nothing else: no NC, FF, or FC counts it, no final vote and no N entry follows from it, and no report contains it, since reports are frozen before it is cast. Late notarizations are retained evidence that a manifest may carry, but only inside an exclusion witness. They give a block published after the boundary the closing-epoch exclusion witness that the overlap exception requires.

C. Attachment and prefix-wide eligibility

Let $d_a(S)$ be the maximum depth of any block for account a retained in checkpoint S. With S_{e-1} installed, a retained tip continues at its own depth (Section C-F), and a *new* epoch-e root starts at depth $d_a(S_{e-1})+1$ on a maximum-depth retained tip. Every parent, a retained tip included, must be *settled for extension* in epoch e: final, held under a notarization lock (as its target or as an ancestor of the target), or complete in epoch e. A tip held by recovery records alone is therefore a parent in epoch e only once the validator holds an epoch-e NC of first votes for it, assembled in its own instance at its own position, which it already has (Section C-F). Final blocks and blocks under a notarization lock need nothing new, because no rival is ever admitted past them. An extension must respect finalized history and every carried lock record on its selected ancestry. Before S_{e-1} is known, first voting may prepare work

from a parent final in the latest closed ledger or in the closing epoch (with its proof), from a maximum-depth tip that the latest closed ledger holds under a notarization lock, or from a complete current-epoch parent. The signed admission-witness digest fixes the anchor and old evidence actually used.

Why a recovery-only tip must be notarized again. Letting a maximum-depth retained tip always be extended would be unsafe. A recovery record of origin $e-1$ on B' is discharged by $\mathsf{XW}_{e-1}(B)$ for a rival B, so B may become final in epoch e on that witness, under the overlap exception or after installation. If a child C' of B' is then admitted without a new notarization of B', a fast certificate for C' finalizes B' as well (Case 5 of Appendix F). Lemma 1 compares supports at one position only, and C''s certificate puts no epoch-e vote at B's position, so nothing else excludes the pair. With the rule, finalizing C' needs $\mathsf{NC}_e(B')$, which $\mathsf{NC}_e(B)$ excludes; Lemma 2 states this in general.

Ordinary early work is provisional: when the predecessor arrives, it is rechecked against the decided frontier, finalized history, and carried lock records, and a tally of fast-final size does not bypass this check. A notarization lock constrains settlement to its branch. A recovery record of origin o on a block B' is *discharged* by a verified exclusion witness $\mathsf{XW}_o(B)$ for a block B that conflicts with B': account exclusion (Lemma 1) then proves that no epoch-o final certificate for B' can exist. The witness is retained. Evidence of any other epoch does not discharge the record, and a block carrying several records is released only when each has been discharged by a witness of its own origin.

The core overlap exception. A block may finalize before S_{e-1} is known only if the verifier checks its entire selected prefix against the latest closed state and proves cross-boundary exclusion at *every unresolved position being finalized*: each such block, including the certificate target, must have a closing-epoch exclusion witness XW_{e-1} and a current-epoch NC_e for the same hash. A closing-epoch finalized prefix is already a safe anchor. Any inherited recovery record bypassed by this path requires the matching-origin discharge witness above; inherited notarization locks cannot be bypassed. We call the bundle of these witnesses, the final certificate, and the discharge witnesses for the bypassed records an *overlap certificate* OC. Its validity is a function of its contents and of the last closed state alone (Lemma 8), so a validator that learns it before installing S_{e-1} and one that learns it after reach the same verdict.

A fresh descendant without its own closing-epoch exclusion witness remains provisional, even if its parent is notarized in both epochs; otherwise disjoint committees could certify different children of that parent. If the predecessor arrives first, nonfinal early work falls back to ordinary rechecking; already finalized overlap work remains in the certified live overlay.

Fresh-child recovery. If a checkpoint retains unresolved tips, the owner can extend a permissible tip with a fresh child; finalizing that child selects its unresolved ancestors rather than reopening their slots, and an undischarged lock record still constrains the branch choice. A tip held under a notarization lock is permissible at once; a tip held by recovery records alone becomes permissible in an epoch once it is notarized there again. Under the termination rules the owner need not act at all: the forked position itself becomes final within two good epochs (Theorem 4). Persistent submission therefore means retrying an omitted block or supplying a fresh, possibly empty, uncontested child of a retained admissible tip.

APPENDIX C
FORMAL EPOCH TRANSITIONS

A. Lagged committees and local boundaries

The account committee for epoch e is $K_e = C_{e-2}$. That outgoing committee decides S_e; the already known successor $C_{e-1} = K_{e+1}$ verifies and installs it. The state S_e derives C_e, whose account service starts in epoch $e+2$. Genesis supplies $S_{-1} = S_G$ and $C_{-2} = C_{-1} = C_G$.

After local duration D, a validator may end epoch e only when no older epoch remains closing. It stops old first and final voting, records all complete eligible old blocks as notarized, freezes its report against the closed S_{e-1}, and opens $e+1$; learning a decided checkpoint also stops any remaining signing for its closed epoch. A validator signs nothing new in an epoch it has left except late notarizations, which stop when it installs the checkpoint, but re-broadcasts what it signed before, so that certificates of the closing epoch can still complete. If closure outlasts D, the next local boundary is deferred while successor voting continues. Boundaries are local events; nothing requires a synchronized stop.

B. Immutable signed reports

Every correct outgoing validator i signs one header $R_i = \mathsf{Sign}_i(\mathsf{ctx}_e, \mathsf{Root}(T_i), \mathsf{Root}(G_i))$, where the context binds the session, closing epoch, predecessor checkpoint, outgoing committee, and reporter identity. A Byzantine identity may equivocate, but a candidate selects at most one of its headers. Let L_i be the reporter's ledger at freeze time. Its canonical tagged set is $T_i = \{(h(B), s_i(B)) : B \in L_i\}$ with $s_i(B) \in \{\mathsf{R}, \mathsf{N}, \mathsf{F}\}$: R is inherited recovery-protected state, N represented notarization without locally observed finality, F finality by inherited checkpoint state or by an explicit account certificate for the block or a descendant selecting it; the strongest status wins. T_i ranges over the whole ledger, but an entry that S_{e-1} already holds with the same status is inherited: it needs no evidence, and reconstruction, the usability check, and $\mathsf{BuildState}$ read only the entries in which T_i differs from S_{e-1}, so a closure costs what the epoch changed, not the history. Each entry also names the block's account position and parent, so that a selection places every block it names without the block body. Let $V_i^{(e)}$ contain the hashes of all blocks for which i released an epoch-e first vote before freezing. The second committed set is

$$G_i = V_i^{(e)} \setminus \{h : (h, \mathsf{N}) \in T_i \text{ or } (h, \mathsf{F}) \in T_i\}.$$

An N or F entry already exposes the vote. An R entry does not: it records what the predecessor carried, not what i signed

in epoch e, so a reporter that first-voted an R-tagged block lists it in both T_i and G_i. Removing every key of T_i from G_i would hide such a fresh vote and let two conflicting fast certificates form (Appendix E).

Correct reporters retain the frozen sets, vote records, and admission evidence; later events change their live view but not the signed report. Late notarizations are released after freezing, so they are in no $V_i^{(e)}$; they create no N entry, no Rule 1 or Rule 2 support, and no Rule 3 promotion.

C. Usable reports

Reconstruction is provided by the convergence service with integrity C1; RAI adds the semantic check. A report is *usable* at j when both sets are reconstructed, every entry is well formed, and its claims are justified by evidence j holds: an R entry matches predecessor recovery state; an N entry has the relevant NC of q first votes or an inherited notarization record; an F entry has valid inherited checkpoint finality, an explicit account certificate, or an overlap certificate; and a G_i membership is backed by i's own signed epoch-e first vote and names no hash that T_i tags N or F (it may name one tagged R). A report naming a block above the first position without its parent, unless the predecessor finalized that block there, is malformed and never selected, so one Byzantine report cannot make every candidate unconstructible. Missing evidence is fetched from retained data; a Byzantine report with unavailable or malformed material stays unusable. Closure needs only $N - f$ usable reports, and at least that many reporters are correct.

D. Eligible candidates

A candidate selects exactly $N - f$ usable report headers from distinct members of K_e, in canonical identity order, forming Q, and commits to an evidence manifest $\mathcal{M}$ listing content-addressed blocks, report reconstructions, the exact supporting vote records including the late notarizations inside exclusion witnesses, overlap certificates, and finite admission and dependency witnesses, all of them retained protocol data. The candidate value is $v_e = (Q, \mu_e, d_e)$ with $\mu_e = H(\mathcal{M})$, $d_e = H(S_e)$, and $S_e = \mathsf{BuildState}(S_{e-1}, Q, \mathcal{M})$. RAI's eligibility predicate, which the consensus service enforces through A2, holds for $v = (Q, \mu, d)$ when Q has $N - f$ usable reports of distinct members of K_e, $H(\mathcal{M}) = \mu$, $\mathsf{BuildState}(S_{e-1}, Q, \mathcal{M}) = S \neq \bot$, and $H(S) = d$, with all contexts, report identities, vote domains, and predecessor references verified. A correct member evaluates $\mathsf{Eligible}_e$ only on reconstructed and validated inputs; a digest establishes neither availability nor validity. The manifest fixes the witness bytes before voting, so the state digest is a function of fixed semantic inputs rather than of certificate arrival order; it is not a license to omit required report material, nor may it promote an N or G_i entry to final because an incidental certificate is present.

E. Deterministic construction

A candidate block is an inherited block, a selected tagged block, or a block named by a selected G_i, with the ancestors and fixed dependencies needed to validate it. Eligibility is checked against the closed predecessor or the overlap evidence retained with its admission, never against the candidate's own certificate, which would make Rule 2 circular; an unexplained speculative path is excluded from new recovery counts. For a block B, let $\mathsf{f}_Q(B)$ mean that a selected F entry names B and its finality witness verifies, and let the fresh reporter support set $U_Q(B)$ contain the selected reporters i with $h(B) \in G_i$ whose epoch-e first vote for B the manifest verifies and is *settled*, that is, names S_{e-1} as its base. An inherited R entry is not itself support; a settled epoch-e first vote for an R-tagged block is, and reaches U_Q through G_i. An early first vote is listed in G_i as well but is not counted: its signer had not installed S_{e-1} and may have voted a block that S_{e-1} dropped.

Final closure. Start with inherited finality and all blocks satisfying f_Q. Take account-ancestor closure and recursively include the explicit finalized-send dependencies required by those final blocks. Call the result $F^\star$.

Rule 1: represented notarization. At each new account position, after excluding conflicts with $F^\star$, retain the unique eligible represented NC target and its ancestry as a notarization lock; same-domain exclusion rules out two conflicting eligible NCs.

Rule 2: settled reporter first votes. Rule 2 runs after Rule 1 has been applied at every position. It retains blocks with their ancestry under recovery records of origin e, the closing epoch, whether or not a block already carries records of earlier origins, in two passes. Eligibility in both passes also excludes conflicts with every record already chosen in this construction.

- *Threshold pass, from the highest new position of the account down.* At a position with no represented eligible NC and no block held by a Rule 1 or threshold record that this construction placed at a higher position, retain the eligible block B with $|U_Q(B)| \geq r$, if there is exactly one. A position held only through an inherited record is not skipped.
- *Keep pass, from the lowest position up.* At a position where no block is final or held by any record, inherited or new, directly or through a descendant, retain the block that S_{e-1} held there if it is still eligible; otherwise the eligible block with the largest $|U_Q(B)| \geq 1$, ties broken by the smaller hash. If no eligible block has a settled reporter vote, the position stays empty.

A formable fast-final block, or a descendant of it, is retained by the threshold pass (Theorem 1). The order matters: Lemma 1 bounds only the rivals at the fast-final block's own position, and a rival of one of its ancestors can have r reporter votes; an upward pass retains that rival and drops the fast-final block (Appendix D-C). Counting settled votes only and preferring the predecessor's block make the retained side change at most once (Lemma 11). The keep pass is needed for termination: a position the checkpoint leaves empty is voted afresh, and an adversary that can split the fresh votes can keep it open (Theorem 4). Counting reporter first votes of any kind, position by position from the lowest, is unsafe (Appendix D-C).

Algorithm 1 $\mathsf{BuildState}(S_{e-1}, Q, \mathcal{M})$

1: Verify the predecessor, selected identities, report roots, and the committed manifest.
2: Take every selected (T_i, G_i) from $\mathsf{Reconstruct}$; verify the material for all required memberships and claims.
3: Form the parent-closed candidate set; classify admission and eligibility from fixed evidence.
4: Compute $F^\star$ by explicit-finality, ancestor, and finalized-send dependency closure.
5: Compute represented NC targets (first votes only) and support sets U_Q (settled first votes only) from every selected G_i, including members that T_i tags R, for every position before pruning.
6: For each inherited recovery record, look for an exclusion witness of the record's origin in $\mathcal{M}$ and discharge that record only. Verify each F claim that rests on an overlap certificate (Lemma 8). Reject claims that conflict with a surviving record.
7: Apply Rule 1 at every position, then Rule 2's threshold pass from the highest position down and its keep pass from the lowest up; stamp each Rule 2 record with origin e; retain surviving inherited records and the ancestor closure of required lock targets, forming R_Q.
8: Apply Rule 3 to obtain P_Q; compute F^+ from $F^\star \cup P_Q$.
9: Retain $R_Q \setminus F^+$ and replay $F^+ \setminus F_{e-1}$ once in canonical dependency order.
10: Return (L_e, Σ_e); Rule 3 has effect only if this candidate is decided.

Inherited state and discharge. Carry every predecessor lock record forward unless explicit compatible finality excludes its block or, for a recovery record of origin o, the manifest contains an exclusion witness $\mathsf{XW}_o(\cdot)$ for a conflicting block. A selected vote or claim for a block that conflicts with a carried record is eligible only if the manifest contains that witness; the record is then discharged in this construction. A block leaves the retained state only when none of its records survives and it is not an ancestor of a block with a surviving record. Discharge is never inferred from a certificate's newer epoch number, and all evidence summaries are computed before any omission. Let R_Q be the surviving inherited unresolved state, selected lock targets, and their ancestor closure.

Rule 3: checkpoint promotion. For each account, starting at the tip of $F^\star$, promote the next block only while it is the unique child retained in R_Q and the manifest verifies a closing-epoch NC of first votes for it; stop at the first fork or block without such an NC. The prefix P_Q is candidate-final until the consensus service decides the candidate. Recovery-only blocks are never promoted. Let F^+ be the ancestor- and finalized-send-dependency closure of $F^\star \cup P_Q$; the output ledger marks F^+ final and retains $R_Q \setminus F^+$.

Replay orders parents before children and finalized sends before dependent receives, choosing the least $(a, v, h(B))$ among independent ready blocks. A cyclic or semantically invalid graph is rejected; Lemma 9 shows that an all-correct usable selection never is.

F. Installing a decided checkpoint

Let a validator's current closed base be S_b with finalized set F_b, and O_i its certified overlay, so that its effective final set is $J_i = F_b \cup O_i$. On an accepted proof π_e for v_e, the validator recomputes S_e from the inputs v_e names, checks $H(S_e) = d_e$, and installs it with final set F_e: $J'_i = J_i \cup F_e$ and $O'_i = J'_i \setminus F_e$. The union is checked for account compatibility, final dependencies, and valid lock discharge; the validator adopts S_e as its base and materializes the live view as $\mathsf{Replay}(\Sigma_e, O'_i)$, with persistent operation identifiers suppressing duplicate application. The base's final set grows monotonically; every overlay block has an explicit account-finality proof for itself or a selecting descendant; and the union extends each base-final account prefix and respects every undischarged lock record.

Committee C_{e-1} holds no second election: it verifies the outgoing committee's proof, reconstructs the state, derives C_e from the canonical Σ_e, and rechecks nonfinal early work, discarding invalidated provisional work without clearing its persisted first vote or terminal state. Locks the checkpoint retains are continued, not reopened: every retained block gets an instance in the open epoch at its own position, a rival at that position is refused while any record on the retained block survives, and under the termination rules of Appendix B-B a rival whose witness discharges the records is admitted but not voted. A first vote in that instance is an ordinary epoch-e first vote; if the block does not finalize, the vote is exposed at the next closure through G_i, and Rule 2 keeps the block: under a record of origin e if r selected reporters voted it, and otherwise through its surviving records or the keep pass, which prefers it. An NC_e assembled in that instance is also what lets a block held by recovery records alone serve as a parent in epoch e (Appendix B-C).

Appendix D
Full Proofs

The arguments concern the core protocol, including the prefix-wide overlap guard, late notarization, the early-vote rule, origin-stamped records, and Rule 3 promotion, and are relative to the services of Appendix A-D: safety uses C1 and A1–A3, termination additionally C2 and A4. Cryptographic guarantees hold except with negligible probability.

Lemma 1 (Account exclusion): In one account instance at most one block has an exclusion witness; in particular at most one has an NC. If an FC or FF for B can be formed in that instance, no conflicting block has an exclusion witness. If an FF for B can be formed, every conflicting block has at most $f + p < r$ supporters.

Proof: Two sets of q supporters intersect in at least $2q - N = f + 1$ identities, one of them correct, and a correct validator supports one block per instance (Definition 1), by a first vote or by a late notarization. An FC for B has q final voters; a correct final voter first-voted B in the same instance, so the same intersection applies against any XW of a rival. An FF has $N - p \geq q$ first voters. For the last claim let $f' \leq f$ be the actual number of Byzantine members. An FF has at least $N - p - f'$ correct first voters, so at most p correct members and f' Byzantine members can support a rival: at most $p + f' \leq p + f$. ∎

Lemma 2 (Notarized ancestry): If an epoch-e NC, FC, or FF for a block D can be formed, then every ancestor X of D is final, or is held under a notarization lock of a decided checkpoint (as its target or as an ancestor of it), or has an epoch-e NC that can be formed.

Proof: Each of the three certificates contains at least $q \geq f + 1$ first votes for D (a correct final voter first-voted the same block in the instance), so some correct validator first-voted D. By the attachment rule of Appendix B-C it did so on a parent P that it held final, held under a notarization lock, or held complete in epoch e, that is, with an $\mathsf{NC}_e(P)$ in hand. Ancestors of a final block are final, and ancestors of a notarization-lock target are retained under that lock, which Rule 1 creates with the target's ancestry and the inherited-state rule never drops while the target is unresolved. In the third case $\mathsf{NC}_e(P)$ can be formed and the argument repeats for P; the chain is finite. ∎

The lemma rests on the re-notarization rule of Appendix B-C: a tip held by recovery records alone is not a final parent. If such a tip could always be extended, the lemma would fail, and with it Theorem 2 (Case 5 of Appendix F).

Proof structure. Theorems 1 and 2 are proved by one induction over closures, from genesis. The step for closure e uses Theorem 2 for finality through epoch e (in Rule 1's exclusion of conflicts with $F^\star$ and in the earlier-epochs case) and Theorem 1 for closure $e-1$ (the closure invariant of S_{e-1}); Theorem 2 up to epoch $e+1$ uses Theorem 1 for closures up to e. Each proof below cites the other only within this order.

Lemma 3 (Late notarization adds no old finality): Every epoch-e NC, FC, or FF consists of first and final votes released by correct signers before they left epoch e, together with Byzantine votes. A correct validator releases a final vote, or records an N entry, only for a block for which it holds an NC of q first votes.

Proof: By Definition 1 and the certificate definitions of Appendix B-B none of the three certificates counts a late notarization, and a correct validator releases no epoch-e first or final vote after leaving e (Appendix C-A). Completeness is defined on NC_e, and notarization and final voting require completeness. ∎

Remark 1: A restarted prototype validator signs its recorded votes again (Appendix G), so a vote recorded but not signed before a crash can be released for the first time after the validator left the epoch, which the lemma's first sentence excludes. The proofs are unaffected: the decision was recorded before the report froze and none follows the freeze, so the report lists the vote, and the first votes behind a final vote are written with its decision, so a report built after a restart still tags the block N. Lemma 4 therefore holds for it, and Theorem 1 counts it whenever a certificate is completed. `RaiCloseRestart` shows that the durable notarization is needed; no model covers signing a vote again.

Lemma 4 (Frozen-report exposure): Let i be a correct reporter for closure e. If i released an epoch-e final vote for B, then T_i tags B with N or F. If i released an epoch-e first vote for B, then T_i tags B with N or F, or $h(B) \in G_i$. This holds whatever other tag B inherits from S_{e-1}, and it persists after the report is frozen.

Proof: A final vote follows durable notarization, which is an N entry unless finality has replaced it by F. For a first vote, $h(B) \in V_i^{(e)}$, and G_i removes from $V_i^{(e)}$ exactly the hashes tagged N or F. Stopping precedes freezing; roots and tags are then immutable, and C1 ensures that what a correct validator reconstructs is what was frozen. ∎

Remark 2: With $G_i = V_i^{(e)} \setminus \mathsf{keys}(T_i)$ instead, the second statement fails when S_{e-1} carries B under a recovery record: T_i tags B with R and the fresh vote appears nowhere. `RaiClose` with `FIXB = FALSE` reaches two conflicting fast certificates this way (Appendix E).

Lemma 5 (Signers hold the witness): Every correct signer of an eligible epoch-e FC or FF for B holds, as retained evidence, an exclusion witness discharging each recovery record of S_{e-1} on a block that conflicts with B.

Proof: An $\mathsf{FF}_e(B)$ consists of settled first votes. A correct signer of one had installed S_{e-1} when signing; the voting rule of Appendix C-F refuses a rival of a retained block unless every record on it is discharged, and the signer's admission witness fixes the witnesses it used, so it holds them. A correct final voter either released its final vote after installing S_{e-1}, with the same check, or before. In the second case the early-vote rule required it to hold $\mathsf{XW}_{e-1}(B)$ for B and every unresolved block of the selected prefix. That witness discharges every record on a rival that closure $e-1$ created, since those have origin $e-1$ (Rule 2 stamps the closing epoch). Records that S_{e-1} inherits from S_{e-2} were known to the voter, because S_{e-2} is installed before the overlap of closure $e-1$ begins (at most one closing epoch per validator), and the overlap guard required the matching-origin witness for each of them. ∎

Definition 2 (Closure invariant): A closed state S_e satisfies the closure invariant if, for every epoch $e' \leq e$ and every block B with an eligible epoch-e' account final certificate that can be formed from released signatures, S_e holds B (i) as final, (ii) under a notarization lock, or (iii) under a recovery record of origin e', directly or as an ancestor of a block so held.

Theorem 1 (Closure preserves latent and late finality): Let S_{e-1} be a valid closed predecessor that satisfies the closure invariant, and let v be any candidate with $\mathsf{Eligible}_e(v)$ and state S_e. Let B have an eligible epoch-e account final certificate that can be formed, possibly completed after reports froze or after the checkpoint was decided, and assume

(W) B conflicts with no lock record carried by S_{e-1}, or every correct signer of the certificate holds, as retained evidence, an exclusion witness that discharges each such record.

Under the early-vote rule, (W) holds for every eligible certificate (Lemma 5); without that rule it can fail (Appendix D-A). Then S_e holds B as final, under a notarization lock, or under a recovery record of origin e, directly or as an ancestor of a block so held; no admissible evidence discharges that hold; and S_e satisfies the closure invariant for all certificates covered by (W). The same holds for every ancestor of B that the certificate

finalizes with it.

Proof: Let Q be the selected reporters, $|Q| = N - f$, and let A be the signer set of the certificate.

Eligibility of B in the construction. If B conflicts with no carried record there is nothing to show. Otherwise, by (W), each correct signer holds an exclusion witness for every carried record that B bypasses. That witness is retained evidence; reconstructing the signer's report material delivers it, the manifest contains it, and the record is discharged at line 6 of Algorithm 1. So B is eligible for Rules 1 and 2 whenever a correct signer is selected, which the counts below guarantee.

Normal certificate. A is a set of q final voters, so $|(Q \cap A) \setminus \mathsf{Byz}| \geq (N - f) + q - N - f = q - 2f = p + 1 \geq 1$. By Lemma 4 each such reporter froze an N or F entry for B. A verified F entry puts B in $F^\star$. Otherwise B is a represented NC target; it is the unique one at its position by Lemma 1, it does not conflict with $F^\star$ by the induction hypothesis of Theorem 2, and Rule 1 retains it under a notarization lock.

Fast certificate. A is a set of $N - p$ settled first voters, so $|(Q \cap A) \setminus \mathsf{Byz}| \geq (N-f)+(N-p)-N-f = N-p-2f = r$. Take these reporters. If any selected report has a verified F entry for B we are in the first case. Else if any has a verified N entry for B, Rule 1 applies as above. Else, by Lemma 4, each of the r reporters has $h(B) \in G_i$ with a verified settled first vote, whether or not T_i tags B with R; hence $|U_Q(B)| \geq r$.

Rule 2's threshold pass works down from the highest position, so we first show that no record placed by Rule 1 or by that pass above B's position is on a branch that conflicts with B. Such a record is on an eligible block D with a represented $\mathsf{NC}_e(D)$ or with $r \geq f + 1$ selected reporters' settled first votes, so some correct validator first-voted D in epoch e. Let $Y \neq Z$ be the blocks of D's and B's branches at the first position where they differ, Z being B or an ancestor of B. The argument of Lemma 2, applied to that correct first vote, shows that Y is final, held under a notarization lock of a decided checkpoint, or has an $\mathsf{NC}_e(Y)$ that can be formed; Lemma 2 for B's certificate shows the same for Z. If either is final or notarization-locked, the other's branch conflicts with a final block, contradicting Theorem 2 through epoch e, or bypasses a notarization lock, which makes B or D ineligible. So both $\mathsf{NC}_e(Y)$ and $\mathsf{NC}_e(Z)$ can be formed, or $\mathsf{NC}_e(Y)$ and $\mathsf{FF}_e(B)$ if $Z = B$, contradicting Lemma 1. Hence when the threshold pass reaches B's position, either a record above holds a descendant of B, and B with it, or the position is not yet held and Rule 2 runs there. No rival has a represented NC, and every rival B' has $|U_Q(B')| \leq f + p < r$, both by Lemma 1; so B is the unique eligible block reaching r, and the threshold pass retains it under a recovery record of origin e. The keep pass runs afterwards, only where nothing is held, and retains no block that conflicts with a record already chosen.

The order is essential. Lemma 1 bounds only the rivals at B's own position. A rival of an ancestor Z of B may have r reporter first votes when Z has only an NC_e, and a pass that worked upward would retain that rival at Z's position before reaching B, and then drop B as conflicting. Appendix D-C gives the run.

No discharge. A notarization lock is never bypassed. A record of origin e on B is discharged only by $\mathsf{XW}_e(B')$ for a conflicting B', which Lemma 1 excludes because an epoch-e final certificate for B can be formed. Evidence of other epochs does not apply to it, and discharging records of other origins on B leaves it in place.

Finality through a descendant. Let X be an ancestor of B that is not final before the certificate. If S_{e-1} holds X final or under a notarization lock, the inherited-state rule keeps it. Otherwise S_e holds X as an ancestor of B, by the cases above, and the question is whether that hold can be discharged. A hold through B ends only when every record on B is discharged or a final block excludes B. A record of origin e on B is discharged only by $\mathsf{XW}_e(Y)$ for the block Y of a conflicting branch at the first position where that branch leaves B's; that position holds B or an ancestor X of B. By Lemma 2, X is final, under a notarization lock, or has an $\mathsf{NC}_e(X)$ that can be formed; the first two exclude the conflicting branch outright, and in the third Lemma 1 excludes $\mathsf{XW}_e(Y)$. A notarization lock on B or finality of B is never undone. If instead the threshold pass holds B through a record of origin e on a descendant D, that record is discharged only by an XW_e for a block that leaves D's branch at some position; D has a correct settled first voter, so, as above, the block of D's branch at that position is final, under a notarization lock, or has an NC_e that can be formed, and Lemma 1 excludes the witness. This step needs Lemma 2: if a tip held by recovery records alone could be extended, an $\mathsf{XW}_e(Y)$ could exist at an ancestor's position, and in Case 5 of Appendix F Y even becomes final.

Earlier epochs. For $e' < e$ the predecessor holds B as final, under a notarization lock, or under a record of origin e'. Finality is inherited. A notarization lock and a record of origin e' are carried forward by the inherited-state rule, since the only ways to drop them are a conflicting final block, excluded by Theorem 2, and an $\mathsf{XW}_{e'}$ of a rival, excluded by Lemma 1.

The argument uses only signatures in A and their intersection with Q, so it does not matter when the certificate is assembled, and unselected or Byzantine signatures cannot remove the correct intersections. ∎

Lemma 6 (Checkpoint-promotion safety): A Rule 3 promoted block cannot conflict with a block that has a closing-epoch exclusion witness, FC, or FF. Hence a decision that finalizes the promoted prefix is compatible with old-domain finality and with core overlap finality.

Proof: Every promoted block has a verified closing-epoch NC of first votes, which is an exclusion witness; Lemma 1 excludes an exclusion witness, and so an FC or FF, for a conflicting block. Core overlap finality for a conflicting successor block requires a closing-epoch exclusion witness at the first divergent position, which is the same contradiction. Promotion stops at retained forks and never promotes recovery-only evidence. ∎

Lemma 7 (Acyclic final dependencies): For any finite set of valid final blocks closed under account ancestors and required finalized sends, parent and send-to-receive edges form a directed acyclic graph.

Proof: Order blocks by the earliest event at which valid final evidence justifies them. Along a parent edge this event cannot become earlier at the child; along a send-to-receive edge it is strictly later at the receive, because admission required the send's finality proof and Rule 3 cannot supply it for the checkpoint under construction. A cycle cannot consist only of parent edges, since slots increase strictly, so it contains a send-to-receive edge and a strict increase around a cycle, a contradiction. ∎

Lemma 8 (The overlap verdict does not depend on installation order): Let closure e be in progress, let B and B' conflict at one position, and let OC be an overlap certificate for B: an exclusion witness $\mathsf{XW}_e(B)$, an $\mathsf{NC}_{e+1}(B)$, an epoch-$(e+1)$ FC or FF for B or a descendant selecting it, and an exclusion witness of the matching origin for every recovery record on B' that S_{e-1} carries, with no notarization lock on B' in S_{e-1}. Then for every candidate with $\mathsf{Eligible}_e$ and state S_e: (a) S_e does not finalize B'; (b) S_e carries no notarization lock on B'; (c) every recovery record that S_e carries on B' is discharged by a witness contained in OC. Consequently a validator that holds OC accepts B as final under the overlap exception before installing S_e if and only if it accepts B by ordinary rechecking after installing S_e, and a later manifest that contains OC verifies an F claim for B against S_e.

Proof: (a) S_e finalizes a block through inherited finality, an explicit closing-epoch account certificate, or Rule 3. Inherited finality of B' contradicts the check of OC against S_{e-1}, which is installed before the overlap of closure e begins. An epoch-e FC or FF for B', and the $\mathsf{NC}_e(B')$ that Rule 3 requires, are excluded by $\mathsf{XW}_e(B)$ and Lemma 1. (b) A new notarization lock needs a represented $\mathsf{NC}_e(B')$, excluded the same way; an inherited one is excluded by hypothesis. (c) A record on B' in S_e is either created by Rule 2 at this closure, with origin e, and then $\mathsf{XW}_e(B) \subseteq \mathsf{OC}$ discharges it; or inherited from S_{e-1}, and then OC contains its discharge witness by hypothesis.

For the consequence: before installation the validator checks exactly the contents of OC against S_{e-1}. After installation, ordinary rechecking tests B against the frontier, the finalized history, and the carried records of S_e; by (a)–(c) these tests pass on the evidence in OC and on no other. The same evaluation is what line 6 of Algorithm 1 performs at closure $e+1$. The lemma is stated for one divergent position; the prefix-wide guard applies it at the first divergent unresolved position of the finalized prefix, as the proof of Theorem 2 does, and the `RaiPrefix` model of Appendix E checks that guard on two positions. ∎

Lemma 9 (Deterministic construction): Assume the predecessor satisfies the closure invariant. For fixed usable selected reports and a valid immutable manifest, $\mathsf{BuildState}$ returns one canonical candidate state at every correct validator. It preserves inherited finality, carries all mandatory surviving records, and applies only explicit account finality plus Rule 3 promotion. All-correct usable inputs yield a finite valid construction.

Proof: By C1 every correct validator reconstructs the same (T_i, G_i) for each selected header, and the manifest fixes witnesses and proof choices; hence explicit finality, discharges, and R_Q are the same everywhere, with lock records compared as sets of (block, strength, origin). Lemma 1, predecessor validity, attachment, and Theorem 1 make the retained finality and records compatible. Rule 2's two passes visit positions in a fixed order, each against the records already chosen, and use only the selected settled votes, the predecessor, and the hash order, so they choose the same records everywhere. Rule 3's tests fix P_Q, safe by Lemma 6; Lemma 7 and the canonical tie-break fix replay. An N entry of a correct reporter is backed by an NC of first votes (Lemma 3), which the manifest can carry, so the entry verifies. An F entry of a correct reporter for a block finalized under the overlap exception is backed by its overlap certificate, which is retained evidence; by Lemma 8 line 6 accepts the claim and discharges every conflicting record of S_{e-1}, so no all-correct selection is rejected on that account. Malformed Byzantine inputs are excluded by usability; all-correct usable inputs yield a finite state. ∎

Theorem 2 (Transfer safety across epochs): Assuming C1 and A1–A3, finalized histories at correct validators are compatible and irreversible across all core-protocol epochs; finalized balances are nonnegative, and each send is received at most once.

Proof: Induct from genesis, jointly with Theorem 1. *Within an epoch*, let two blocks be final through epoch-e certificates for D and D' whose branches conflict, and let $X \neq X'$ be their blocks at the first position where the branches differ. Conflicting finality need not be visible at a single position: either certificate may finalize X or X' only as an ancestor. By Lemma 2 each of X, X' is final through older evidence, held under a notarization lock, or has an NC_e that can be formed. Two NC_e at one position contradict Lemma 1. A notarization lock on X is never bypassed: a correct validator that installed it refuses X' and its descendants, and one that had not accepts D' only under an overlap certificate, whose $\mathsf{XW}_{e-1}(X')$ contradicts the closing-epoch NC behind a new lock (Lemma 1) and whose check against the latest closed state refuses an inherited one. Finality through older evidence is the boundary case. At a boundary, A2 ensures that the decided v_e is eligible, so by Theorem 1 it preserves every formable old final branch and by Lemma 6 its promotion is compatible; A1 and A3 ensure that every correct validator installs the state of the same v_e, which by Lemma 9 is the same state. For overlap, the first divergent unresolved position on an early successor-final prefix has a closing-epoch exclusion witness for the successor block by the prefix-wide guard, whose late notarizations add no old finality that Theorem 1 would have to preserve (Lemma 3); a conflicting closing-epoch certificate or a conflicting Rule 3 promotion would require a conflicting NC, contradicting Lemma 1; and by Lemma 8 correct validators that hold the same overlap certificate reach the same verdict whether they verify it before or after installing S_e, so they agree on which overlap blocks are final and not only on the compatibility of what they have finalized. Matching-origin discharge and inherited records remain enforced after installation, and the base/overlay union with applied-operation

identifiers prevents rollback and duplicate application. Replay checks balances and finalized-send dependencies. ∎

Lemma 10 (Correct reports become usable): Under the data service, finite relevant admitted work, fair processing, and C2, every correct report eventually becomes usable at every correct validator.

Proof: A correct reporter stops old first and final voting, so its old vote set and roots are finite, and its report is well formed: every N entry rests on an NC of first votes (Lemma 3), every F entry on inherited finality, an account certificate, or a retained overlap certificate, and every G_i member on its own first vote and on no hash tagged N or F. By C2 every correct validator eventually reconstructs (T_i, G_i). The data service delivers the blocks, ancestry, signed votes, late notarizations, and overlap certificates behind its claims, and fair processing completes the finite validation. A withholding Byzantine reporter can make only its own report unusable; $N - f$ correct reports remain. ∎

Theorem 3 (Handoff termination): Assume the data service, fair processing, C2, A4, and recurring prepared windows with $N - f$ responsive correct members of K_e long enough for reconstruction, candidate preparation, and the consensus service's rounds. Every enabled handoff is eventually decided and, with eventual retained-data delivery, installed by correct successors.

Proof: By Lemma 10, $N - f$ correct reports become usable at every correct member, which by Lemma 9 can then construct an eligible candidate; the canonical selection order makes correct members that hold the same usable reports construct the same one. A4 then decides some eligible value in a sufficient window; A3 lets successors verify it, and retained authorizing data let them recompute and install the state, which enables the next deferred boundary. ∎

Theorem 4 (Account continuation and forked positions): Assume handoff termination, available final-send dependencies, and recurring common-open ready intervals. (a) An owner following the retry-or-fresh-child policy of Appendix B eventually finalizes each target of a finite valid uncontested continuation, provided each required new position has such an interval, and the finalized targets eventually enter a checkpoint. (b) Let (a, v) be a position whose parent is final when epoch e starts and at which some valid block is available. If epochs e and $e+1$ are *good*, some block at (a, v) is final by the end of epoch $e+1$, whatever the owner and the Byzantine validators do. An epoch is good if every correct member of its committee installs the predecessor checkpoint, receives every block and vote that a correct validator sends at the position, and casts every first and final vote the rules allow it before freezing its report. (c) Hence the owner of a forked account, once it signs no further conflicting blocks, finalizes its continuation as in (a) after the forked position is final; and at most one block at the position ever becomes final.

Proof: (a) In a ready interval, $N - f \geq q$ correct first votes create an NC and the same validators can final-vote; after they install the predecessor their first votes are settled, so an available $N-p$ fast quorum finalizes in two stages. An attempt interrupted by a boundary is preserved by Theorem 1; an omitted block can be retried, and a uniquely retained notarized target may finalize at the handoff.

(b) Consider epoch e. Every correct member installs S_{e-1} and first-votes at the position before freezing; where the position is contested for it, it votes only after installing S_{e-1} and then, if S_{e-1} holds an unresolved block K there, only K. Suppose no block at the position is final at the end of e. Then the position is contested at every correct member: if S_{e-1} holds a block there, every correct member installed it; otherwise, if all correct members first-voted the same block B, then B had $N-f \geq q$ correct first votes, every correct member final-voted it after installing S_{e-1} (nothing there refuses it), and B would be final; so correct members voted two blocks, and each has seen both.

Next, S_e holds an unresolved block K' at the position or finalizes one. If a selected report represents an eligible NC_e, Rule 1 locks it, and Rule 3 promotes it unless another block stays retained there. Otherwise, a record of S_{e-1} that survives keeps its block, or the threshold pass of Rule 2 keeps a block there, directly or through a descendant. If neither does, the keep pass reaches the position: it retains S_{e-1}'s block there if it is still eligible, and otherwise the eligible block with the most settled reporter votes. One has a vote: the selected reports contain at least $N - 2f \geq 1$ correct reports, each listing its member's epoch-e first vote unless it tags it N or F, and in a good epoch every correct first vote is settled. Every block at the position is eligible there, since nothing final and no record, inherited or new, holds the position.

Now consider epoch $e+1$. The position is contested at every correct member, so each first-votes there only after installing S_e, and then only K', unless a rival is final, which would end the argument. The cross-epoch rule does not interfere, since it lapses at installation. K' is retained, so every correct member admits its instance, and in a good epoch it receives $N - f \geq q$ correct first votes. The same members final-vote K' after installation, which the termination rules allow because K' is the block S_e keeps, and q correct final votes make it final. A block whose parent position is final needs no complete parent, and the re-notarization rule concerns only children of K'.

(c) Once a block at the forked position is final, the account's later positions are uncontested if the owner signs no further conflicting blocks, and (a) applies; a fresh child of the final block is the owner's continuation. At most one block at the position becomes final by Theorem 2.

The rules of (b) only withhold votes, except Rule 2's choice, which retains every block that may be fast-final (Theorem 1); so they do not weaken safety. By Lemma 11, the side the checkpoints keep at the position changes at most once on the way. Without them the position can stay open forever (Appendix D-B). Normal finality takes three delivery stages after broadcast and an $N - p$ settled first-vote quorum two; a sufficient account interval budget is $D/(1+\rho) > \kappa_A + R_A + 3\delta$, where κ_A bounds opening skew and R_A readiness after the last opening, including predecessor closure. ∎

Lemma 11 (One change of side): Under the termination rules, let a decided checkpoint S_k hold a block K at a position whose parent is final. Then S_{k+1} holds K there, or a conflicting block K' that has an eligible final certificate or a notarization lock in S_{k+1}; every later checkpoint then holds K'. Hence the block that the checkpoints hold at such a position changes at most once.

Proof: A final block and a notarization lock are never released, and a block with an eligible final certificate stays held by Theorem 1; so it suffices to consider S_k holding K by recovery records only, and S_{k+1} holding neither K' case. A correct validator that installed S_k casts no first vote at the position or above it on a branch that bypasses K's records, unless the bypassing block is final at it, which is the first case (*checkpoints decide*). So every settled epoch-$(k+1)$ first vote of a correct validator on a branch through the position is on K's branch. A threshold record at or above the position needs $r \geq f+1$ selected settled votes, one of them correct, so it holds K; a Rule 1 record of a rival is excluded by assumption. If a record of S_k on K survives, S_{k+1} holds K. Otherwise the keep pass reaches the position with nothing held there and retains S_k's block K, which is still eligible: it conflicts with no final block and no notarization lock by assumption, and with no other record by the above. ∎

The proof uses both parts of Rule 2. With the hash order in place of the predecessor's block, a witness that discharges K's records lets a Byzantine listing move the position, and `RaiRecoverMany` reaches two changes of side (`Many2_routeA_down_hashpick_flip`). With every first vote counted, simulation found no second change (`Many2_routeA_down_allvotes_flip`): *no early votes at a contested position* already keeps early votes away from a position once an installed checkpoint holds it. With every first vote counted and the downward order, the safety checks found no violation either, in 32,503 random traces and in every run of up to 15 steps (`Many2_routeA_any_down_allvotes_safe` and `_E3_bfs`), and the prototype finalizes forks left after the load about twice as fast with it (Appendix G). We count settled votes because the proof then does not rely on that rule, and because the recipes of Table III that count every first vote in another order change sides twice.

A. Early votes against a carried lock

Without the early-vote rule, hypothesis (W) of Theorem 1 is not cosmetic. The following run violates the theorem when the early-vote rule is dropped, whatever the definition of the second set. `RaiHandoff` finds it with `RULE = FALSE`. Take $N = 6$ with validator 6 Byzantine, and conflicting blocks B, B'.

1) Epoch e: validators 1, 2, 6 first-vote B'. Validators 3, 4, 5 cross their boundary without voting at this position.
2) Before S_e is known, 3, 4, 5 first-vote B in epoch $e+1$ and late-notarize it in e. Validator 6 signs both votes for B and shows its late notarization to validator 1 only.
3) Closure e selects reports 1, 2, 3, 4, 6. Rule 2 gives B' a recovery record of origin e.
4) Validators 3, 4, 5 install S_e without holding $\mathsf{XW}_e(B)$; their early work on B is invalid in their view. Validator 1 installs S_e, holds $\mathsf{XW}_e(B) = \{3,4,5,6\}$, discharges the record and first-votes B in $e+1$. Now $\{1,3,4,5,6\}$ is an $\mathsf{FF}_{e+1}(B)$ that can be formed.
5) Closure $e+1$ selects reports 2, 3, 4, 5, 6. The only correct holder of the witness is not selected, the manifest has no $\mathsf{XW}_e(B)$, the votes for B are an unexplained speculative path, and S_{e+1} retains B' only.
6) In $e+2$ validators 2, 3, 4, 5, 6 first-vote the retained B': $\mathsf{FF}_{e+2}(B')$. Validator 6 then publishes its signatures; B has an FF_{e+1} whose eligibility against S_e follows from $\mathsf{XW}_e(B)$.

With a committee change the same failure needs no late notarization: four new members early-vote B, the old committee's first votes for B plus one withheld Byzantine vote are the exclusion witness, and the second closure sees four unexplained votes. The cause is that the eligibility of an early first vote can depend on a witness its signer never held. The early-vote rule removes the dependency: in step 4 the set $\{1,3,4,5,6\}$ is not a fast certificate, because three of its votes are early, and validators 3, 4, 5 final-vote B only once they hold $\mathsf{XW}_e(B)$, in which case their reports deliver it to the next closure.

Two alternatives fail. Retaining both blocks at the second closure keeps the one-round path but can stall a contested position permanently after a committee change. Counting only early votes of members of the closing committee is unsafe: the committee-change run above defeats it.

B. Why forked positions need the termination rules

Without the termination rules of Appendix B-B and Rule 2's keep pass, a forked position can stay open forever although every epoch is good. Take the first run of Case 5 with the re-notarization rule, which refuses the owner's child C' until B' is notarized again. In each later epoch validators 3, 4, 5 first-vote B again: before installing the checkpoint the cross-epoch rule allows re-voting the same block, and after it the old witness $\mathsf{XW}_e(B)$ still admits B. Validators 1 and 2 vote B', and validator 6 withholds the one vote either side needs. Neither block is notarized, the next checkpoint keeps B' again, and the owner cannot help: C' waits for B'. Nor could a different child rule help: in the second run B may already be final when the owner publishes C', and validators 1 and 2 cannot tell the two runs apart. Recovery cannot rest on the owner extending the kept block either, since that route is unsafe.

Each rule removes one ingredient. With *checkpoints decide*, validators 3, 4, 5 stop voting B after installing the checkpoint; with *no early votes at a contested position*, they do not re-vote it before (without this rule alone, `RaiResolve` finds the stall in 20 steps: in two good epochs the correct validators re-vote their old sides early, two against one, and the Byzantine validator withholds the third vote); and with Rule 2's keep

pass, a checkpoint never leaves a voted position empty, which an adversary could otherwise split again. `RaiResolve` (Appendix E) finds the stall with the rules off, also when the position starts uncontested, and checks with them on that two good epochs finalize the position.

C. Rule 2 from the lowest position

Consider a Rule 2 that keeps, position by position from the lowest, the block with the most selected reporters' first votes of any kind, ties by hash. It violates Theorem 1 and Theorem 2. `RaiRecoverMany` finds the following run in 40 steps (`Many2_allup_hole_tight`), and its mirror image without a script (`Many2_allup_nc`, 57 steps). Take $N = 6$ with validator 6 Byzantine, conflicting blocks A_1, A_2 at position 7, and a child C_i of each A_i at position 8. The owner signs C_i only once a checkpoint keeps A_i.

1) Epoch 1: validator 1 first-votes A_1, validator 2 first-votes A_2, and nobody else votes at position 7. Closure 1 keeps A_2 under a recovery record of origin 1: the Byzantine report lists A_2, two votes against one.
2) Epoch 2, before installing S_1: validators 2 and 4 first-vote A_2 early, 3 and 5 first-vote A_1 early; each has seen one block only, and S_0 keeps nothing at position 7. 3 and 5 also late-notarize A_1 in epoch 1, so with 1's vote and 6, A_1 has an XW_1.
3) Validator 1 installs S_1 and first-votes the kept A_2: with 2, 4, and 6 that is an $\mathsf{NC}_2(A_2)$, so under the re-notarization rule C_2 may now be voted. Validators 1, 2, 4, 5 install S_1 and first-vote C_2; with 6 that is an $\mathsf{FF}_2(C_2)$, which finalizes A_2 too. Validator 5 voted A_1 at position 7 and C_2 at position 8, which no rule forbids, since a vote for a child is not a vote for its parent.
4) Closure 2 selects reports 2, 3, 4, 5, 6, none of which shows A_2 notarized. $\mathsf{XW}_1(A_1)$ discharges S_1's record on A_2. Rule 2 at position 7 counts 3, 5, and 6 for A_1, which is r, and 2 and 4 for A_2; it keeps A_1 under a record of origin 2. At position 8, C_2 now conflicts with that record and is dropped, although $\mathsf{FF}_2(C_2)$ can be formed.
5) Epoch 3: every correct validator installs S_2 and votes only A_1 at position 7. Validators 1, 2, 3, 4 first-vote it settled; with 6 that is an $\mathsf{FF}_3(A_1)$. A_1 and A_2 are both final.

The proof's fast-certificate case bounds rivals only at B's own position by $f + p$, but A_1 is a rival of C_2's parent, which had only an NC. Counting settled votes only removes this run, since 3's and 5's votes for A_1 are early, but not its shape: an owner that signs C_1 at once lets C_1 be voted in the fork epoch itself, where nothing is kept yet. In epoch 1, 2, 3, 4 first-vote A_1, which is notarized with 6; 1 and 5 vote A_2; 1, 2, 4, 5 vote C_1, an $\mathsf{FF}_1(C_1)$ with 6; closure 1 selects 1, 2, 3, 5, 6, where A_2 has the r settled votes of 1, 5, and 6, and keeps A_2 (`Many2_routeA_any_up_safe`, 20 steps). The threshold pass from the highest position removes both runs: C_i has the settled votes of r selected reporters and is retained first, with its parent, and Rule 2 skips position 7. Ancestor support, under which a first vote for a child also supports its parent, would remove them as well, since no correct validator could then support A_1 at position 7 and a child of A_2 at position 8 in one epoch; we chose the order because it changes only the construction.

Appendix E
The TLA⁺ Models

The nine modules (the eight models of the main text, which counts `RaiRecover` and `RaiRecoverMany` as one), their TLC configurations, a run script, and the TLC outputs (including the violating traces) are in the `tla/` directory of the artifact. `RaiClose`, `RaiOverlap`, and `RaiHandoff` take one account position with two conflicting blocks X and Y, $N = 6$, $f = p = 1$, five correct validators and one Byzantine validator whose signatures are available for every block, vote kind, and epoch and whose report is chosen adversarially subject to the usability checks of Appendix C-C. `RaiPrefix` and `RaiRecovery` take two positions and are run with $N = 4$ ($f = 1$, $p = 0$) and $N = 6$. `RaiResolve` takes one position over three epochs and is run with $N = 4$. The consensus service is the nondeterministic choice of one eligible candidate from any $N - f$ usable reports. TLC explores all interleavings; a `VIEW` identifies validators up to permutation. The runs of Table II used TLC build 2026-10-06 (two workers) and TLC 2.20 (eight workers, `RaiPrefix`).

Invariant names. `NoConflict` and `Agreement`: no two conflicting blocks are final (Theorem 2 at the position). `ClosureThm1`, `ClosureThm1a`, `ClosureThm1b`: Theorem 1 for the respective closure, stated without hypothesis (W); some archived outputs name them `Thm64`, `Thm64a`, `Thm64b`. `OneFinal`: a checkpoint finalizes at most one block per position. `SameVerdict`: Lemma 8. `UsableReports`: Lemma 10 once every validator has left the epoch. `WitnessAdmissible`: line 6 of Algorithm 1 at the next closure can verify a correct F claim against an inherited lock on its rival. `EarlyCompatible` (`RaiPrefix`): every block finalized early is compatible with every formable epoch-1 certificate. In `RaiRecovery`, `NoConflict` counts a parent as final when its child is. `Probe*`: reachability probes, expected to be violated, which show that the invariants are not vacuous.

`RaiClose`: three voting epochs, two closures. Votes are cast only after the voter's predecessor checkpoint is decided, so there is no early voting, late notarization, or overlap finality. Each closure runs BuildState with Rules 1–3, inherited records with origins, and matching-origin discharge; the manifest's discharge set is any subset of the dischargeable records that contains those a selected correct vote relied on. Switch `FIXB`: G_i lists fresh votes for R-tagged blocks (`TRUE`, Appendix C-B) or omits every key of T_i (`FALSE`). Abstractions, each of which only adds behaviours: a validator's activity in different epochs is carried by separate identities, so a validator may still sign in epoch e while epoch $e + 1$ votes; a validator that observes its NC releases its final vote in the same step; reports are forgotten once their closure is decided.

RaiOverlap: closing epoch, open epoch, one closure. Validators cross their local boundary one by one. After it, and before installing S_1, a validator may first-vote in epoch 2 under the cross-epoch guard and then also signs a late notarization in epoch 1. A block may finalize before S_1 is known under the overlap exception, or by rechecking against S_1 after installation. Switches: `NCSPLIT` (a validator records NC and final-votes only on an NC of q first votes; off, late notarizations count too), `DISCHARGE` (a recovery record is discharged by a conflicting epoch-1 exclusion witness; off, never), `LATEEV` (late notarizations are admissible manifest evidence for discharge; off, they are "never construction input"). Finality decisions are recorded globally rather than per validator: `ovf` for the overlap exception at a validator that has not installed S_1, `pof` for rechecking at one that has. Closure 2 is not built; its acceptance condition for the case of interest is `WitnessAdmissible`.

RaiHandoff: early voting, two closures, changing committee. Epoch 1 is voted and closed by K_1, epoch 2 by K_2; the Byzantine validator sits in both. A member of K_2 may first-vote in epoch 2 before installing S_1 (an early vote); if it is also in K_1 and supported nothing else in epoch 1, that vote comes with a late notarization in epoch 1. The second set of Appendix C-B, the NC split, matching-origin discharge, and late votes as evidence are built in. `RULE = FALSE` drops the early-vote rule: a fast certificate counts all first votes and a validator final-votes on its epoch-2 NC alone. `RULE = TRUE` is the early-vote rule: a fast certificate counts only first votes cast after the signer installed S_1, and an early final vote is released only by a validator that holds the epoch-1 exclusion witness. A block is final when its certificate can be formed from released signatures and it is valid against S_1, through the overlap bundle or by rechecking with discharge. The builder of S_2 holds a discharge witness only if a selected correct reporter relied on it; otherwise the adversary decides, which models a withheld signature. Further abstractions: correct reports carry no F tag for an observed certificate (covered by the other two models); a claim that conflicts with a surviving record is ignored rather than invalidating the selection, which only adds checkpoints; epoch 3 is not modelled, so safety after S_2 is checked as Theorem 1 for S_2.

RaiPrefix: the prefix-wide guard on two positions. One account with a parent P and a rival P' at position 1 and two children C_1, C_2 of P at position 2. Epoch 1 is closing and voted by K_1, epoch 2 is open and voted by K_2; one Byzantine validator sits in both committees, and the two committees are identical, disjoint, or share k members. Members of K_1 cross their boundary one by one. S_1 is never installed within the model, so every epoch-2 first vote is early: early votes form no fast certificate, and an epoch-2 final vote is released only under the overlap guard. A member of both committees that first-votes x in epoch 2 also signs a late epoch-1 notarization for x, unless it first-voted at that position in epoch 1, and the cross-epoch rule stops it from first-voting a different block where it supported one in epoch 1. The closure and BuildState are left out: early finality is checked against latent epoch-1 finality, that is, against any formable epoch-1 FC or FF. Switches: `GUARD = "prefix"` requires XW_1 and NC_2 for every unresolved block on the path to the anchor, `GUARD = "target"` checks only the finalizing block; `FRESHCHILD = TRUE` requires a child to have its own XW_1, `FALSE` lets NC_2 for the child plus a parent with NC_1 and NC_2 suffice. Abstractions that only add behaviours: votes are signer sets, checks read formable certificates, finality is global, and members of $K_2 \setminus K_1$ may vote in epoch 2 from the start.

RaiRecovery: building on a recovery-locked block. One account with conflicting blocks X and Y at position 1 and a child of each at position 2, C_X on X and C_Y on Y. Epoch 1 is voted by K_1 at position 1 and closed as in `RaiHandoff`; epoch 2 is voted by K_2 at both positions, early (before installing S_1, with late notarization, the cross-epoch rule, and the early-vote rule) or settled. Before installation a child is first-voted only on a parent complete in epoch 2 or final in epoch 1; after installation, on a parent final in S_1, or complete in epoch 2 and admissible against S_1, or, with `EXEMPT = TRUE`, retained by S_1 (any retained tip may be extended). Early final votes need the prefix-wide overlap bundle with a fresh child witness. A final child finalizes its parent, and `NoConflict` counts that. Abstractions: nobody votes at position 2 in epoch 1, finality is global, and closure 2 is not built, so the model checks Theorem 2 within epoch 2 and against S_1. Run with $N = 4$ and $N = 6$.

RaiCloseRestart: restarts before the report freezes. `RaiClose` with a validator that restarts before freezing its report and keeps its final vote; it runs with $N = 6$ and with $N = 4$ ($f = 1$, $p = 0$). Switch `RESTART`: the notarization survives (`durable`); it is lost, and the report lists the first vote without an NC tag (`lose`); the report waits until the notarization can be assembled again without the Byzantine validator (`hold`); or, in addition, until frozen reports of others rule out a normal certificate (`release`). `NoStuck`: no state in which every correct validator still to report can never freeze while the Byzantine validator stays silent. With `durable` safety and `NoStuck` hold for both sizes. With `lose`, safety fails once $p + 1$ validators restart: both sides final after 13 steps at $N = 4$ and 19 at $N = 6$. `hold` and `release` are safe, but `hold` gets stuck with one restart (7 and 10 steps), and `release` once all $f + p + 1$ correct final voters of a block restarted (8 and 12 steps). Rule 2 keeps a side here by the most reporter votes of any kind, which approximates the keep pass. The configurations and outputs are in `tla/`.

RaiResolve: termination at a forked position. One account position with conflicting blocks X and Y, three voting epochs, two closures, one committee of $N = 4$ ($f = 1$, $p = 0$). Voting follows Appendix B: one first vote per epoch, the cross-epoch rule, admission with matching-origin discharge, early final votes only with the closing-epoch witness, settled votes only on the fast path; a validator leaves epoch e only with S_{e-1} installed. The Byzantine validator signs first votes for either or both blocks in any epoch, at any time, or withholds

them, reports adversarially within the usability checks, and the manifest discharges any dischargeable subset. Validators see the two blocks when the adversary delivers them. Each epoch is flagged good or not; in a good epoch no correct validator leaves before every correct validator has installed the predecessor, seen both blocks, and cast every first and final vote the rules allow. `TwoGood`: two consecutive good epochs make some block final. Switch `RULES`: the two termination rules and Rule 2's keep pass. `Spec23` starts after an arbitrary fork epoch (any epoch-1 votes and Byzantine signatures, anything seen, any checkpoint closure 1 can build, installed by nobody yet) and makes epochs 2 and 3 good; `Spec` explores every pattern of good epochs and is run by simulation. Abstractions: no late notarizations and no committee change; a formable final certificate counts as finality, whose validity the safety models check; the Byzantine validator signs no final votes, which could only complete a certificate; a correct report tags a block N only after its final vote, which can only hide notarizations.

Rule 2 keeping a side (`MAXKEEP`). `RaiClose`, `RaiOverlap`, `RaiHandoff`, and `RaiRecovery` take a switch `MAXKEEP` under which Rule 2 also keeps a side where no block reaches r: the block with the most reporter first votes of any kind, ties by hash. Configurations with the suffix `_maxkeep` set it on; the keep pass of Appendix C-E itself is checked by `RaiRecover` and `RaiRecoverMany`.

`RaiRecover` and `RaiRecoverMany`: four epochs, three closures, recovery. One account with a fork at position v and a child of each forked block at $v+1$; $N = 6$, $f = p = 1$; four voting epochs and three closures, one committee. Each validator crosses its own boundary and installs checkpoints on its own, so a first vote is settled or early. Built in: the cross-epoch rule, late notarization, the early-vote rule, the prefix-wide overlap guard, the second set of Appendix C-B, Rules 1–3 with origins, and matching-origin discharge; Rule 1 runs at every position before Rule 2. `RaiRecover` has two blocks A, A' with free epoch-1 votes. `RaiRecoverMany` has $B \in \{2, 3\}$ rivals $A_1, \ldots, A_B$; validator $i \leq B$ first-votes A_i in epoch 1 and the others nothing; validator 6 may deliver any block to anyone at any time, and a correct validator may vote any admissible block whenever it may vote. The owner signs a child once a decided checkpoint keeps its parent or the parent is final (`OWNER = "policy"`), or signs every block at the start, with free epoch-1 votes (`"any"`). Switches: `SETTLED` (Rule 2 counts settled first votes only); `PICKRULE` (`none`: only a block alone reaching r; `hash`: else the most votes, ties by hash; `prev`: the keep pass); `R2ORDER` (`down`: the threshold pass from the highest position; `up`: from the lowest); `IMPLIED` (ancestor support: a first vote for a child also supports its parent); `ATTACH` (`strict`: *checkpoints decide* at later positions too; `permissive`: a witnessed rival may be voted); `RENOTARIZE`; `NOEARLY` (*no early votes at a contested position*, the half that reads the installed checkpoint); `BYZ` (`all`: every Byzantine signature available; `release`: Byzantine first votes released one at a time, final votes never); `READY` (the good epochs). Invariants: `NoConflict`, `ClosureThm1` for each closure, `OneKept`, `FlipBound` (Lemma 11), `KeptOrFinal` (every checkpoint keeps or finalizes a block at v), `RecoveredMany` (a block at v is final when no correct validator or closure can move), `YieldNext` (a checkpoint of the epoch of an early certificate keeps its block), and probes. `RaiRecoverScenarios` and `RaiRecoverManyScenarios` add action constraints that confine a run to the roles of a trace; a constraint only removes behaviours, so a violation found under one is a run of the full model.

Configuration names: `allup` is a Rule 2 that counts every reporter first vote and keeps, from the lowest position, the block with the most, ties by hash, with re-notarization; `routeA` is Rule 2 of Appendix C-E (settled votes, keep pass) with re-notarization; `_down` the downward threshold pass, otherwise upward; `_any` the forking owner; `patch` ancestor support with settled votes and the keep pass, without re-notarization, and `none`, `hash`, `perm` that variant with no keep rule, with the hash order, or with permissive attachment; `allvotes` and `hashpick` Rule 2 with every first vote counted or with the hash order in place of the predecessor's block. Suffixes: `safe` (`NoConflict`, `ClosureThm1`, `OneKept`), `flip`, `kof`, `live`, `two23`, `gap24` (`RecoveredMany` with epochs 3–4, 2–3, 2–4 good), `early` (early finality under the overlap rule, with `YieldNext`). Abstractions that add behaviours: finality from formable certificates; complete delivery and voting any admissible block; a correct report may omit an N tag; the manifest discharges any dischargeable set containing the records a selected correct vote relied on; a child is published as soon as a checkpoint keeps its parent; `NOEARLY` leaves out the seen-two-blocks half. Abstractions that remove behaviours: correct reports carry no F tag for an observed certificate; with `BYZ = "release"` no Byzantine final vote is released, so the recovery runs miss attacks that need one; the owner policy, so safety against a forking owner is checked only by the `_any` runs; scripted epoch-1 votes in `RaiRecoverMany` unless `OWNER = "any"`; a block held only by the keep pass is never promoted; one committee. No configuration is exhaustive: runs are random simulation (depth 150, two workers, stopped after 5 to 25 minutes) or breadth-first search stopped at a time cap.

Table II lists every configuration. The `Overlap_noSplit` run shows that counting late notarizations toward an NC damages usability, hence termination, not safety: with the split off the other four invariants hold over the full state space. The `Prefix` runs show that the two shortcuts one might take in the overlap rule are unsafe as soon as the committees are not identical: checking only the finalizing block (`target`) fails with two shared members at $N = 4$ and with three at $N = 6$, and letting a child inherit its parent's witness (`nofresh`) fails with disjoint committees and with one or two shared members; the prefix-wide guard with a fresh witness per child (`fixed`) passes every $N = 4$ configuration. The `Handoff` runs with one and four shared members and the breadth-first `Prefix6`

runs of the prefix-wide guard were stopped before completion. The `Recovery` runs find the retained-tip exemption unsafe for every pair of committees at $N = 4$ and by simulation at $N = 6$, and the re-notarization rule safe in every $N = 4$ configuration and, at $N = 6$, in every run of up to 24 steps (the search was stopped at depth 25); the three probes show that the precondition (a final block against a recovery lock on its rival) and owner recovery, also after a validator voted the dropped side, are reachable. No model covers more than two positions, receive dependencies, other committees, or any N other than 4 and 6; only `RaiRecover` and `RaiRecoverMany` reach a third epoch and a third checkpoint, and they check Theorem 1 with finality through a descendant by simulation and, for Rule 2 with two rivals and three epochs, by a breadth-first search to depth 17 (Table III). The `Resolve` runs show the stall without the termination rules, from the start and from an arbitrary fork epoch, and with them that two good epochs after any fork epoch finalize the position, exhaustively for $N = 4$ and three epochs, and in every pattern of good epochs that half a million random runs reached; one good epoch is not enough. With `MAXKEEP` on, every safety configuration of these four models passes.

Table III lists the `RaiRecover` and `RaiRecoverMany` runs. `RaiRecover` checks ancestor support, an alternative to re-notarization: without the re-notarization rule a fresh child of a kept block finalizes both sides (Case 5 again, now across three closures; `Scen4_trace1`); with ancestor support or re-notarization and the keep pass no violation was found; counting every first vote, or the hash order, lets the kept side change twice; either of them, or permissive attachment, can leave a forked position open; and one good epoch is not enough. `RaiRecoverMany` checks Rule 2 itself. With a Rule 2 that counts every reporter first vote from the lowest position (`allup`), `ClosureThm1` and `NoConflict` fail with two rivals (Appendix D-C), and so do `FlipBound` and `KeptOrFinal`; the three-rival runs found no safety violation in the time given. With settled votes and the keep pass but the upward order, `ClosureThm1` still fails against a forking owner. With Rule 2 of Appendix C-E no violation was found: safety against a forking owner, `FlipBound`, `KeptOrFinal`, recovery with epochs 3–4, 2–3, or 2–4 good, and early finality under the overlap rule, where S_1 may keep a rival of the early-final block and S_2 then keeps that block; with the hash order in place of the predecessor's block `FlipBound` fails, and with every first vote counted it held in the time given. The upward configurations without `_any` are included for comparison; `patch` is the ancestor-support alternative, checked with two rivals only. A breadth-first search of Rule 2 with two rivals, a forking owner, and three epochs found no violation of `NoConflict`, `ClosureThm1`, `OneKept`, or `FlipBound` in any run of up to 17 steps (21.3 M distinct states, stopped after 3 h); the same search with the upward order finds `ClosureThm1` violated after 15 steps, so the depth reached covers the shortest failure of that order. Everything else is simulation; Theorem 1 with the downward pass and Lemma 11 rest on the proofs above.

Appendix F
Scripted Regression Cases

The random Byzantine voter of the benchmark harness does not find the runs of Appendix E. The following scripted cases were derived from the violating traces; $N = 6$, validator 6 is Byzantine, B and B' conflict.

Case 1 (second set). Epoch 1: validators 1, 2, 3 vote B'; 4, 5 vote B; 6 reports B only; 3's report is not selected. Epoch 2: 1 to 4 first-vote the retained B, nobody assembles the NC. Release the epoch-1 NC for B' after the second close. Expected: the second checkpoint carries B with an origin-2 record; B finalizes; B' never does.

Case 2 (discharge). Before the boundary 1, 2, 6 vote B'. After it 3, 4, 5 vote B in the open epoch and late-notarize it; 6 joins both. Install the checkpoint at 1 and 2 before they see B's certificate. Expected: every validator finalizes B, and the next closure builds.

Mixed NC. One validator still in the closing epoch receives three first votes and one late notarization for its block. Expected: it does not record N; its report is usable.

Case 3 (early votes). The run of Appendix D-A. Expected: B has no fast certificate; 3, 4, 5 final-vote B only if they hold the witness, and then the second checkpoint finalizes B. Without the early-vote rule this run ends with both blocks final.

Case 3, new members. Epoch 1: 1, 2, 3 vote B; 4, 5 vote B'; 6 reports B' only and withholds its vote for B. Four members who were not in the old committee early-vote B. Same expectation as Case 3.

Case 5 (re-notarization). Epoch e: 1, 2, 6 first-vote B' at position 7; 3, 4, 5 cross their boundary without voting there. Before S_e is known, 3, 4, 5 first-vote B in epoch $e + 1$ and late-notarize it in e; 6 signs both for B, so $\mathsf{XW}_e(B)$ and $\mathsf{NC}_{e+1}(B)$ hold. Closure e selects 1, 2, 3, 4, 6 and keeps B' under a recovery record of origin e. 3, 4, 5 final-vote B, either after installing S_e, where $\mathsf{XW}_e(B)$ discharges the record, or before it under the early-vote rule once 6 shows them its signatures; with 6, B is final. The owner publishes C' on B' at position 8, and 1 to 5 first-vote it after installing S_e: a fast certificate for C' finalizes B' too. `RaiRecovery` with `EXEMPT = TRUE` finds a shorter run without early votes: in epoch e, 1 and 2 vote X and 3 votes Y ($N = 4$), the checkpoint keeps Y on the Byzantine report, and after installing it 1, 2, 3 first-vote both X (whose witness discharges Y's record) and C_Y. Expected under the re-notarization rule: C' is refused at its first vote, because B' has at most the epoch-$(e + 1)$ votes of 1, 2, and 6 at position 7; in a run where nobody voted B, B' is notarized again and C' finalizes both.

Case 6 (termination). The stall of Appendix D-B: after the checkpoint keeps B', validators 3, 4, 5 re-vote B before installing it and vote B again after installing it on its old witness; validator 6 withholds. Expected: under the termination rules, 3, 4, 5 do not first-vote at the contested position before installing the checkpoint and vote only B'

after it, which is notarized and final in the next good epoch; with a three-to-three split in the reports the checkpoint keeps the side with the smaller hash.

Case 7 (Rule 2 order). The run of Appendix D-C. Expected: closure 2 retains C_2 and A_2 by the threshold pass, and in epoch 3 nobody votes A_1. With a forking owner: closure 1 retains C_1 and A_1.

Case 4 (prefix). Epoch 1: P is notarized; C_1 has first votes from 1, 2, 3. After the boundary, members of a committee sharing two members with the old one first-vote C_2 early and late-notarize it. Expected: C_2 does not finalize early without its own epoch-1 witness, and no validator finalizes C_2 while an epoch-1 certificate for C_1 can form.

`RaiClose` with `FIXB = FALSE` prints Case 1 step by step, `RaiOverlap` with `DISCHARGE = FALSE` prints Case 2, `RaiHandoff` with `RULE = FALSE` prints both Case 3 runs, and `RaiPrefix` with `FRESHCHILD = FALSE` or `GUARD = "target"` prints Case 4, `RaiRecovery` with `EXEMPT = TRUE` prints the short form of Case 5, `RaiResolve` with `RULES = FALSE` prints Case 6, and `RaiRecoverMany` with `R2ORDER = "up"` prints Case 7. In the prototype, Cases 1, 2, Mixed NC, 3, and 4 exist as unit tests of the construction and of the voting rules, Case 5 as unit tests of the attachment rule (a child of a recovery-only tip is first-voted only once the tip is notarized in the child's epoch, and at once under a notarization lock), and Case 6 as unit tests of the termination rules (a witnessed rival of a kept block gets an instance but no vote; a contested position waits for the checkpoint; Rule 2 keeps the side with the most reporter votes, ties by hash). Case 7 exists as unit tests of the construction (a child with r settled reporter votes is kept before a rival of its parent with as many; three early first votes lose to one settled vote; the keep pass prefers the predecessor's block over a rival with more votes) and of the voting rule (a child and a grandchild of a witnessed rival of a kept block earn no vote); the benchmark harness's Byzantine validator votes at random and does not play these scripts, and a live harness with a scripted Byzantine validator that withholds named votes was not built. We consider this the main gap between the models and the measurements.

APPENDIX G
PROTOTYPE NOTES

A. Departures from the protocol

First, the manifest commits to signed votes and marks the closing-epoch witness of every overlap certificate its evidence proves: the closing-epoch exclusion witness, the current-epoch NC, a matching-origin witness for every record of the last closed state S_{e-2} the block bypasses, and a parent finalized in S_{e-1} or itself certified. The marks are a function of the evidence, the selection, and the two closed states, so a validator recomputes them and refuses a manifest that marks any other set. Blocks and ancestry are placed by the report items, and the witnesses that discharge records of the predecessor checkpoint are re-derived by every validator from the manifest's votes rather than bound to the claims they serve. Second, a first vote names as its base the state hash of the checkpoint its signer had installed, d_{e-1} for a settled vote and d_{e-2} for an early one, rather than a digest of the per-block admission evidence; a settled vote counts toward a fast certificate only at a validator that installed the checkpoint it names, and a validator that has not yet installed it holds the vote and counts it at installation. With these two departures no vote binds the witnesses its signer held, so a proposer could leave an exclusion witness out of its manifest, or simply lack it, and the construction would then drop the claims that need it. The prototype closes this in the report. An F entry for a block that bypasses a record of S_{e-1} is justified only with the witnesses that discharge those records. A reporter that final-voted a block it tags N lists that final vote in G_i when the block bypasses a record of S_{e-1}, and such a record is justified only with the same witnesses; the report is otherwise not usable, and a manifest that selects it without them is refused. A correct validator final-votes a bypassing block only while it holds them: before installing S_{e-1} through the overlap rule, afterwards through matching-origin discharge. So for a final certificate that can still form, the $p + 1$ correct final voters in any selection (Theorem 1) force the witnesses into the manifest, line 6 of Algorithm 1 discharges the records, and Rule 1 retains the block. First votes need no such rule. After installing S_{e-1} a correct validator first-votes no block that bypasses a surviving record unless the bypassing block is final at it (Appendix B-B), and then its F entry carries the witnesses; an early first vote counts toward no fast certificate and no Rule 2 support, and the construction leaves it out while the record survives. A Byzantine reporter that lists a final vote without witnesses makes only its own report unusable. The rule changes a report only where a final-voted block bypasses a record; the runs of the main text were made before it was added. Third, retained vote records keep a fixed cap of four epochs, since the reports of the next three epochs may still cite an epoch's late finality, and beyond it only the support that may still discharge a surviving recovery record of its origin at its position. The handoff evidence, an epoch's frozen reports and their reconstructions, is released once members of the successor committee carrying $N-f$ of its weight acknowledge the checkpoint; a member acknowledges once the decided state is written and synced, empty checkpoints included, and unreleased epochs are kept up to sixteen, a forced drop being logged (none occurred in any run). Fourth, durability rests on the host's `fsync`: first, final, late, and close-election votes, frozen reports, overlap evidence, the epoch origin, and decided checkpoints are synced before any signature that depends on them leaves the process, but on the macOS host `fsync` hands the data to the drive without forcing its cache. A stricter mode issues `F_FULLFSYNC` after every signing write (about 4 ms each). Power loss was not simulated; the restart runs kill processes and test crash consistency only. A vote's record holds what it says, written before it is signed, not its signature. A restarted validator therefore signs its recorded first and final votes again, in the epochs it left too, and serves them on

request: the same identity, block, kind, and epoch, so the new signature conflicts with nothing, and every such vote is one the validator's report lists. The first votes behind a final vote go into the same transaction as its record, each stored once and only as many as the certificate needs, and come back on a restart with the notarization they prove. Fifth, a position whose committed first votes are split so that no block can be notarized any more is not carried into the new epoch at the boundary; if the checkpoint then retains neither side, the position is voted afresh, and if it retains one side, that side is continued. Sixth, the *pause* transition used as a baseline in the evaluation is a configuration switch of the same code: under it no instance of epoch $e + 1$ is started before S_e is decided locally, so the overlap rule and late notarization never apply; everything else, including reports, the construction, and installation, is unchanged. Seventh, a position counts as contested when any instance at its root holds two candidate blocks or the latest decided checkpoint retains an unresolved block there; a block forked at a different parent is not seen as contested until a checkpoint retains it. Eighth, an evaluation switch turns fast finalization certificates off; the one-host committee-change runs of Appendix G-E, with four seats, use it. Ninth, the Byzantine validator of the evaluation is the same node built with an evaluation-only feature: it signs a second vote for the sibling of every fork block it votes for, sends a report with a corrupted residual root to half of the validators, and, leading the first round of every close, proposes two values derived from two orders of the usable reports, voting for the second at once.

B. Implementation notes

The following mechanisms are not part of the protocol but are what the prototype needs to meet the services' assumptions under load; none changes a rule.

Reports and evidence. A report is laid over the shared state S_{e-1} and holds only what the epoch added, upgraded, or removed; its root is kept incrementally, and the checks and the construction read the changes alone. A validator records, for every block it votes for, the parent that block names, for as long as any slot state that voted the block survives, so that its residual records place the block; a report naming a block without its parent is malformed and never selected. A node's own signed votes enter its vote processing on a lane of their own, past the queue that network votes and the vote cache's replays share, so every vote a report lists is in its signer's vote records and is served to any validator that asks for it as evidence. A proposal carries the signature of every report it selects, as the decided value does with its report headers: a validator authenticates a version of a Byzantine report it does not hold and reconstructs it next to the one it holds, keeping at most four versions per reporter. Manifest chunks are requested from the proposal's sender. An epoch whose handoff evidence was released is remembered, so a peer's repeated report does not recreate it, and a repeated copy of a proposal is not checked again once the value is held or the epoch decided.

Close rounds. Close-round votes bypass the per-channel inbound queue, and requests for a close round's statements take the inbound queue's priority lane and a lane of the request aggregator, so a validator that lags on account positions still completes its closes. Proposals are checked one at a time, the newest round's first, and a round waits from the moment its leader's proposal is queued.

Lagging validators. A validator one close behind its peers fetches the decided value and state by the certificate's value hash from any holder; it asks for the first rounds' statements of a close that has been open for 2 s, ends its epoch once members holding more than f of the weight are seen voting in later epochs, and, while its epoch has no election, asks once per epoch for that epoch's close. A validator that falls behind is flooded with live votes and can drop evidence replies, which delays its readiness for a close.

Retained blocks. A late instance's discard rolls a block, and what was built on it, back from the ledger. The validator keeps the bodies of these blocks, forces a retained one back in from them when a checkpoint keeps it, and answers other validators' requests for it, so a block a checkpoint keeps is always held by some ledger. Carried instances whose lock cannot continue are discarded at installation. An instance first-votes a valid candidate when the block it saw first is not valid, and every validator continues the kept block once its ledger holds it.

Request pacing. Across regions a reply can take longer than the repeat intervals that suit one host, so a validator does not repeat a request for votes before its reply could arrive; the harness sets that floor to the longest round trip it measures (311 to 316 ms in the runs of the main text). An instance this validator has not first voted in, of an epoch whose predecessor checkpoint it has not installed, does not start asking, since the other validators withhold the same votes. Without this, thousands of such instances asking several times a second overfill the request aggregators, the replies crowd the nearer validators, and those drop close votes, so the close that would release the instances stalls. A validator asks a report's evidence of the reporter first and then backs off, backs off a solicitation that brought nothing and a repeated broadcast of an instance's winner, and a terminated instance does not ask for statements it holds.

Epoch boundaries. The start of the run reaches the validators at different times, so their epoch origins differ by up to about a second. An epoch that a validator ends early, because members holding more than f of the weight are voting in later epochs, keeps its boundary: the next epoch runs to the following boundary, so no epoch is cut short.

Durable state and membership. The epoch origin T_0, the genesis committee, the decided checkpoints, and the close votes are durable records; decided states and evidence are kept in a second LMDB environment, and deletions run in bounded transactions, so synced signing does not wait behind them. Committee members are known by membership, not by current weight, and committee derivation waits for the frontier blocks it reads.

Limitations. Every measured block is a representative change: sends and receives occur only in the setup and in the weight moves of the committee change, so no run exercises the finalized-send dependencies of BuildState under load. No run has more than seven validators, and how the close scales with N (it reconstructs $N - f$ reports and checks q signatures per claim) was not measured. With an earlier build one validator could fall 18 to 158 s behind after the burst of votes at an epoch switch, mostly the new member. Its voter read the ledger for every election without a first vote while holding the elections lock, so past some number of elections its scans ran back to back, vote processing got a few percent of its time, its elections stayed open, and its requests reached 0.3 to 12 million against about 50,000; closes that waited for its report slowed down. The build of the main text holds the lock per 64 elections and puts close votes first. On five seeds of the hardest cell the closes went from 42 to 158 s to 8 to 15 s, and across the 72 RAI and pause runs of the main text no RAI validator entered an epoch more than 3.8 s after the median one. Every RAI and pause validator also caps itself at 5,000 asked instances per second, active elections first. Under overload a signed vote can be dropped by every receiver; until its signer supplies it again, its report is unusable at those validators, which delayed closes by up to 110 s in runs with an equivocating validator. With a committee change in every epoch and eight node processes on one host, a rotated-in validator falls far behind its peers at 1,000 blocks/s and above, in RAI and pause alike, and those runs do not settle in time; the single committee change of the main text, with six processes, settles at 2,000 blocks/s.

C. Counting settled first votes

Rule 2 counts settled first votes only. During the load a checkpoint is installed 2 to 6 s into the 8 s epoch that follows it, so most first votes for a fresh fork are early; where a checkpoint finds no settled vote and nothing to keep from its predecessor, it keeps nothing, and the position is voted again and waits one more epoch. On one host (six processes on a MacBook Air, $f = 1$ and $p = 0$ on six members, so that five of six certify, 48,000 blocks at 2,000 blocks/s) with 10% forks the first checkpoint after the load leaves 396 to 529 such positions empty, and the last open block becomes final up to 17 s after the load. The same prototype with every first vote counted, on the same three seeds, leaves none empty, finishes within 4.1 to 6.2 s, and confirms 1,499 to 1,531 non-fork blocks/s against 1,405 to 1,545. Blocks without forks are unaffected. Appendix D explains why the protocol counts settled votes nonetheless.

D. RAI at Nano's quorum

RAI's normal path needs four of six votes and Nano's five. To separate the quorum from the protocol's other rules, 18 further six-region runs use six seats with $f = 1$, $p = 0$ and the fast path switched off, so that a block needs five of six first votes and five of six final votes: the cells and seeds of the main text's matrix without the committee change, three runs each (`run_sweep.py -set final-wan-2000` with `-no-fast-path -committee-p 0 -committee-seats 6`). All 18 settled with identical ledgers, and no block was finalized in one round. Table IV gives the medians. At equal quorum RAI's median latency is within 14 ms of Nano's and its 95th percentile is higher in every cell, by 33 to 410 ms; the lower latency of the main text's RAI therefore comes from its smaller quorum and its fast path. RAI still confirms 1,704 and 1,744 blocks/s with 10% forks, where Nano confirms 946 and 740, and it finalizes every fork. With forks and a Byzantine validator its slowest close takes about 15 s, since every certificate now needs all five honest validators.

E. Committee change

The equal-weight committee model selects the heaviest N representatives from the finalized state, so committee identities change when weight moves. The harness runs extra representatives outside the committee and, at the start of a chosen epoch k, moves a member's whole balance to a holder delegating to a standby representative; the committee derived from epoch k, used from epoch $k + 2$, has the standby in the member's place. The load generator sends the blocks that move weight to every validator, as it does the load. The main text's committee change (its Figure 8) runs across seven regions: six seats with $f = p = 1$, a seventh validator in Toronto as the standby, and Montréal's balance moved to it (`run_sweep.py -set final-wan-2000`). The six members hold equal shares of the voting weight (the genesis representative, a member, keeps about a share) and the standby a sixty-fourth of a share, so for Nano, which counts weight, 67% still takes five of the six members' votes after the move, as in its cells without the change. On one host, six processes on a MacBook Air (Apple M1, eight cores, 16 GiB) over loopback, with six validators and four seats ($f = 1$, $p = 0$, no fast path) and two members replaced in epoch 0 at 2,000 blocks/s with RAI, pause, and Nano, three runs each (`run_sweep.py -set rotation`), the prototype behaves the same; the runs with a Byzantine member use `-set rotation-byz`, which moves the weight of two correct members so that the Byzantine member keeps its seat. Every validator derives the same members for every epoch.

F. Restart, durable signing, and handoff

These runs are on the host of Appendix G-E. Every node logs each signature it releases to a file before sending it. The harness kills validators with `SIGKILL`, either at a fixed time into the load, while elections are running, or when a given close starts, restarts them on their data after a chosen downtime, and afterwards checks every logged signature of every node: no identity may have signed two blocks in one first-vote or final-vote instance or two values in one close round. Fork5 with two restarts, one at a close and one mid-load, settled twice with every checkpoint decided identically on all six validators and no conflicting signature; the restarted validators rejoined through checkpoint catch-up and follow-ahead.

Durable notarization costs little. In five paired runs per cell on this host (no forks and 10% forks at 2,000 blocks/s, 10% forks with a Byzantine validator at 1,000, $f = p = 1$, 45,000 blocks), it adds about 200 bytes per block per validator and raises the average signing write from 0.9–1.2 to 1.1–1.5 ms; the median difference within a pair is under 3 blocks/s, and latency moves both ways within the spread between runs. Five restart runs with two validators killed and kept down 10 s found no conflicting signature and restored every notarization.

Synced signing costs latency, not goodput. Fork5, six validators, 45,000 blocks at 2,000 blocks/s, two runs per setting: without sync 1,765 and 1,820 non-fork blocks/s at p50 173 and 139 ms; with `fsync` 1,802 and 1,688 at 282 and 310 ms; with `F_FULLFSYNC` 1,763 and 1,777 at 409 and 304 ms. The evaluation of the main text runs with `fsync`.

Handoff with a crashing departing member: a member replaced at the rotation of epoch 1 is killed when the close of epoch 2, the last it decides, starts, and stays down 30 s; eight validators, 800 blocks/s, 20,000 blocks, 5% forks. Without and with a Byzantine member every run settled with every checkpoint decided identically (epochs 0 to 4 and 0 to 5), the restarted member caught up the epochs it missed and installed them, every epoch's handoff evidence was released only after $N - f$ durable acknowledgements, no epoch was dropped before release, and no signature conflicted (340 and 335 non-fork blocks/s; the client counts the outage).

G. Reproduction

The prototype, the harness, the TLA$^+$ models, and every run record (the per-run `result.json` with the client's per-second series, its latency histograms by publication second, and the nodes' epoch events, the node configurations, and the logs) are in the artifact. The six-region matrix of the main text is produced from a controller VM by

```
python3 tools/rai/run_sweep.py <dir>
-set final-wan-2000
-blocks 45000 -reps 3 -reps-from 1000
-arm rai=<bin>:-timeout=600:-hosts=<file>
-arm pause=<bin>:-timeout=600:-hosts=<file>:
-epoch-transition=pause
-arm develop=<develop bin>:-no-epochs:
-timeout=600:-hosts=<file>
```

where the hosts file lists one validator address per line in representative order, and `tools/rai/gcp` describes the fleet. A funded ledger per setup comes from `-snapshot-dir`, the VMs are Spot instances, and a sweep ends at the first RAI or pause run that does not settle. Each run in the record has its seed, node hash, and client hash. The 108 runs come from three records. The RAI and pause runs and Nano's committee-change runs are `gcp-matrix3-45k-2026-11-06`: RAI and pause on the node with the voter-lock fix and the request cap (hash `8a7c1763`), Nano on `478d1750`, with equal member weights. Nano's cells without the change are `gcp-matrix2-45k-2026-11-05`, on the same Nano node; the weight change affects only setups with a standby. Eleven runs of the first record in which the client stopped publishing for 4 to 5 s with at most 1,000 blocks in flight, waiting on a write to one validator, were repeated with the same seed and a client that writes to each validator separately (`gcp-matrix4-45k-2026-11-06`, client `8777c487`): RAI repetition 1 with 10% forks, repetitions 1 and 3 without forks and with a Byzantine validator, repetition 2 without forks and repetition 3 with 5% forks, both with the change and a Byzantine validator; pause repetition 2 with 10% forks and the change, repetition 1 with 5% forks and a Byzantine validator; Nano repetition 1 without forks with the change, and repetitions 3, 3, and 2 without forks, with 5%, and with 10% forks, all with the change and a Byzantine validator. The figures use the repeats, which publish in every second of the load but one. The same seed does not give the same run: the repeats differ from the runs they replace in close times by up to a factor of two. The one-host committee change is produced with `-set rotation` and `-set rotation-byz`, `-rate 2000 -blocks 48000 -reps 3`, with `RAI_FAST_PATH=0` in the environment of the RAI and pause nodes. Every arm's binary directory holds `rsnano`, a `rsnano-byzantine` built with the `rai_byzantine` feature, and the generator, all in release mode with the `rai_protocol` feature; Nano is the develop branch with the fork-signature check (`develop-fork-memo`). The sets seed the generator identically for every system (`-seed`), run the Byzantine node (`-byzantine-node equivocate,conflicting,equivocate`) as the first leader of every close (`-byzantine-leads-close`), and keep the nodes running after settling until no forked position is open (`-fork-grace`). One point alone is `tools/rai/run_gate_b.py <dir> -rate 2000 -fork-percentage 10`. The flags `-restart PR:close:E`, `-restart PR:at:S`, `-restart-down-ms`, `-audit-votes`, `-signing-sync`, `-prs`, `-standby`, and `-rotation E:FROM>TO` drive the runs of Appendices G-E and G-F. The figures of the main text are regenerated with `tools/wan_figures.py sections/eval-data/wan <matrix4> <matrix3> <matrix2>`, which takes each run from the first record that has it. The models are checked with `tla/run.sh <module> <config>` and the scripts in `tla/`.

References

[1] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and efficient asynchronous broadcast protocols," in *Advances in Cryptology – CRYPTO 2001*, ser. LNCS, vol. 2139. Springer, 2001, pp. 524–541.

TABLE II

TLC RUNS. TIMES ARE WALL-CLOCK ON TWO CORES; *EIGHT CORES; †SIX WORKERS. SIM IS RANDOM SIMULATION (A VIOLATION FOUND BY SIMULATION IS A REAL TRACE; NO VIOLATION WOULD PROVE NOTHING).

Configuration	Result	Time
`Close_FALSE`	`ClosureThm1` violated after 19 steps	35 s
`Close_FALSE_conflict`	`NoConflict` violated after 23 steps	2 min
`Close_TRUE`	no error; 651,632 distinct states	3 min
`Overlap_fixed`	no error; 5,217,510 distinct states	14 min
`Overlap_noCross` (no cross-epoch guard)	no error; 16,314,028 states	15 min
`Overlap_noDischarge`	`SameVerdict` violated after 15 steps	1 min
`Overlap_noLateEv`	`WitnessAdmissible` violated after 14 steps	1 min
`Overlap_noSplit`	`UsableReports` violated after 9 steps	10 s
`Overlap_noSplit_safety`	other four hold; 5,485,444 states	14 min
`Overlap_probe1`, `_probe2`	probes violated, as intended	$<$ 1 min
`Handoff_same_norule`	`ClosureThm1b` violated after 22 steps	11 min
`Handoff_disjoint_norule`	`ClosureThm1b` violated after 23 steps	15 min
`Handoff_same_rule`	no error; 8,347,138 states	15 min
`Handoff_disjoint_rule`	no error; 3,707,876 states	17 min
`Handoff_same_noCross`, `_disjoint_noCross` (rule, no cross-epoch guard)	no error; 21,898,422 and 3,707,876 states	17, 7 min
`Handoff_overlap3_rule`	no error; 121,053,272 states	2 h 18 min*
`Handoff_Probe*`	probes violated, as intended	$<$ 5 min
`Prefix4_same_{fixed,target,nofresh}`	no error; 278,534 states each	10 s*
`Prefix4_disjoint_fixed`, `_target`	no error; 1,622,276 states	1.7 min*
`Prefix4_disjoint_nofresh`	`NoConflict` violated after 12 steps	5 s*
`Prefix4_shared1_fixed`, `_target`	no error; 3,960,746 states	3.5 min*
`Prefix4_shared1_nofresh`	`NoConflict` violated after 12 steps	8 s*
`Prefix4_shared2_fixed`	no error; 2,434,078 states	1.7 min*
`Prefix4_shared2_target`	`NoConflict` violated after 13 steps	19 s*
`Prefix4_shared2_nofresh`	`NoConflict` violated after 14 steps	12 s*
`Prefix4_*_probes`, `Prefix4_shared2_Probe*`	probes violated, as intended	$<$ 2 s*
`Prefix6_disjoint_nofresh_sim`, `Prefix6_shared3_target_sim`, `_nofresh_sim`	`NoConflict` violated (simulation)	$<$ 1 s*
`Prefix6_same_probes`, `Prefix6_shared3_probes`	probes violated, as intended	14 s, 3.7 min*
`Prefix6_same_fixed`, `Prefix6_shared3_target` (BFS)	stopped at a cap: 13.4 M and 35.8 M distinct states, no violation	–
`Prefix4_{same,shared2,shared1}_noCross` (no cross-epoch guard)	no error; 1,172,954, 6,388,260 and 6,464,176 states	42 s, 5.1, 7.4 min
`Prefix6_same_noCross_sim`, `Prefix6_shared3_noCross_sim`	no violation in 2.27 M and 2.47 M traces (simulation)	30 min each
`Recovery4_same_exempt`	`NoConflict` violated after 16 steps	6 s†
`Recovery4_{disjoint,shared1,shared2}_exempt`	`NoConflict` violated after 16 steps	$<$ 1 min†
`Recovery4_same_fixed`	no error; 734,980 states	17 s†
`Recovery4_disjoint_fixed`	no error; 755,978 states	42 s†
`Recovery4_shared1_fixed`	no error; 3,337,378 states	2.5 min†
`Recovery4_shared2_fixed`	no error; 3,922,566 states	2.2 min†
`Recovery4_same_Probe*`	probes violated, as intended	$<$ 5 s†
`Recovery6_same_exempt_sim`	`NoConflict` violated (simulation)	$<$ 1 s†
`Recovery6_same_ProbeRecovery`	probe violated (simulation)	$<$ 1 s†
`Recovery6_same_fixed` (BFS)	unfinished (stopped after 75 min): 87.5 M distinct states to depth 25, no violation	–
`Close_TRUE_maxkeep`	no error; 278,404 states	1.1 min†
`Overlap_fixed_maxkeep`	no error; 4,959,763 states	6.3 min†
`Handoff_same_rule_maxkeep`, `_disjoint_rule_maxkeep`	no error; 5,862,017 and 2,686,030 states	9.5, 10.6 min†
`Recovery4_{same,disjoint,shared1,shared2}_fixed_maxkeep`	no error; 0.6–3.2 M states	$<$ 3 min†
`Resolve4_norules_sim`, `_norules_23_sim`	`TwoGood` violated (simulation)	$<$ 2 s†
`Resolve4_rules_sim`	no violation in 493,465 random runs (88 M states)	15 min†
`Resolve4_rules_23`	no error, no deadlock; 4,635,660 states, depth 33	8.8 min†
`Resolve4_rules_23_earlykept` (early vote for the kept block allowed)	`TwoGood` violated	4 min
`Resolve4_rules_23_noContested`, `_noR10` (R10 off; with `_noR10` the cross-epoch rule too)	`TwoGood` violated after 20 steps	8, 31 min
`Resolve4_rules_23_ProbeOneGood`, `_ProbeGood23`	probes violated, as intended (10 and 23 steps)	$<$ 2 min†

TABLE III

TLC RUNS OF RAIRECOVER (FIRST BLOCK) AND RAIRECOVERMANY (SECOND BLOCK). NONE IS EXHAUSTIVE: `SIM.` IS RANDOM SIMULATION, BFS A BREADTH-FIRST SEARCH STOPPED AT A TIME CAP. TIMES: FOUR SIMULTANEOUS RUNS OF TWO WORKERS EACH ON EIGHT CORES; THE TWO `_E3_BFS` RUNS USED FOUR WORKERS AND RAN ALONE. CONFIGURATION NAMES ARE EXPLAINED IN THE TEXT.

Configuration	Result	Time
`Recover4_off_{patch,base}_safe`	`NoConflict` violated after 44 / 57 steps (sim.)	< 5 s
`Scen4_trace1`, `_trace1_exact` (BFS)	`NoConflict` violated after 20 / 26 steps	2 min
`Recover4_off_renot_safe`	no violation in 29,129 traces (sim.)	20 min
`Recover4_committed_{safe,flip,fork}`	no violation (sim.)	10–25 min
`Recover3_committed_safe`, `Recover3_all_prev_safe` (BFS)	no violation to depth 19; 9.2 M, 8.8 M states (cap)	30 min
`Recover4_all_unsettled_flip`, `Scen4_trace3_flip`	`FlipBound` violated after 43 / 36 steps (sim.)	< 10 s
`Recover4_all_hash_flip`	`FlipBound` violated after 45 steps (sim.)	3 s
`Scen4_{unsettled,hash,perm}_fork`	`Recovered` violated after 59 / 60 / 65 steps (sim.)	< 20 s
`Recover4_all_prev_{safe,flip,fork,two,two23}`	no violation (sim.)	5–25 min
`Recover4_all_prev_fork` (BFS)	no violation to depth 19; 2.2 M states (cap)	30 min
`Scen4_novotes_live`	`Recovered` violated after 49 steps: one good epoch is not enough	2 s
`Many2_allup_hole_tight` (scripted)	`NoConflict` violated after 40 steps (Appendix D-C)	1 min
`Many2_allup_safe`, `_allup_nc`	`ClosureThm1` violated after 32, `NoConflict` after 57 steps (sim.)	6, 3 min
`Many3_allup_safe`, `_allup_nc`	no violation in 6,113 and 7,323 traces (sim.)	10 min
`Many{2,3}_allup_flip`, `_allup_kof`	`FlipBound` after 35 / 41, `KeptOrFinal` after 38 / 37 steps (sim.)	< 30 s
`Many{2,3}_allup_{live,two23,gap24}`	no violation (sim.)	10 min
`Many{2,3}_allup_any_up_safe`	`ClosureThm1` violated after 21 / 32 steps (sim.)	< 5 min
`Many{2,3}_routeA_any_up_safe`	`ClosureThm1` violated after 20 / 24 steps (sim.)	< 2 min
`Many{2,3}_{paper,routeA}_any_down_safe`	no violation (sim.)	10 min
`Many{2,3}_routeA_down_{flip,kof,live,two23,gap24,patch}`	no violation (sim.)	10 min
`Many{2,3}_routeA_down_early_s1`	probe violated, as intended: S_1 keeps a rival of an early-final block	1 s
`Many2_routeA_down_hashpick_flip`	`FlipBound` violated after 35 steps (sim.)	3 s
`Many2_routeA_down_allvotes_flip`	no violation in 37,326 traces (sim.)	10 min
`Many2_routeA_any_down_E3_bfs` (BFS)	no violation to depth 17; 21.3 M states (cap)	3 h
`Many2_routeA_any_up_E3_bfs` (BFS)	`ClosureThm1` violated after 15 steps	18 min
`Many2_routeA_any_down_allvotes_safe`	no violation in 32,503 traces (sim.)	30 min
`Many2_routeA_any_down_allvotes_E3_bfs` (BFS)	no violation to depth 15; 7.1 M states (cap)	1 h
`Many{2,3}_routeA_{safe,flip,kof,early,live,two23,gap24}`	no violation (sim.)	10 min
`Many2_patch_{safe,flip,live,two23,early}`	no violation (sim.)	10 min
`Many2_none_live`	`RecoveredMany` violated after 60 steps (sim.)	8 s
`Many2_hash_flip`, `_hash_kof`	`FlipBound`, `KeptOrFinal` violated after 41 / 24 steps (sim.)	< 10 s
`Many2_perm_live`	`RecoveredMany` violated after 76 steps (sim.)	53 s
`Many*_probe*`, `Recover4_all_prev_probe*`	probes violated, as intended	< 10 s

TABLE IV
RAI AT NANO'S QUORUM (SIX SEATS, $p = 0$, NO FAST PATH), NANO, AND THE MAIN TEXT'S RAI ($f = p = 1$) ACROSS SIX REGIONS AT 2,000 BLOCKS/S: MEDIANS OF THREE RUNS.

	No Byzantine			One Byzantine		
Forks	0%	5%	10%	0%	5%	10%
Median latency (ms)						
RAI, Nano's quorum	549	538	541	543	540	540
Nano	535	529	533	529	535	533
RAI, $f = p = 1$	390	385	400	393	391	400
95th-percentile latency (ms)						
RAI, Nano's quorum	774	938	1,126	739	1,018	1,077
Nano	592	612	828	595	608	1,044
RAI, $f = p = 1$	483	527	571	508	538	721
Non-fork blocks confirmed per second						
RAI, Nano's quorum	1,888	1,792	1,704	1,880	1,830	1,744
Nano	1,928	1,754	946	1,912	1,770	740
RAI, $f = p = 1$	1,954	1,858	1,750	1,945	1,846	1,747
Slowest close of a run (s)						
RAI, Nano's quorum	4.8	4.4	4.4	8.3	15.1	15.2